

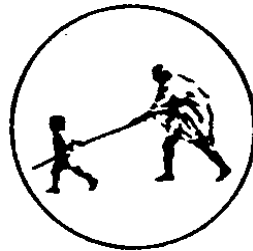
An Internship Report on

“YouTube Channel Performance And Engagement Analytics Dashboard”

BY

Rajlaxmi Naresh Kalaskar

Under the Guidance
Of
Mr. H. U. Joshi



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Mahatma Gandhi Mission's College of Engineering, Nanded (M.S.)

Academic Year 2025-26

An Internship Report on

**“YouTube Channel Performance And
Engagement Analytics Dashboard”**

Submitted to

**DR. BABASAHEB AMBEDKAR TECHNOLOGICAL
UNIVERSITY, LONERE**

for fulfillment of the requirement for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

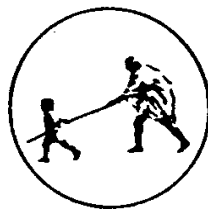
By

Rajlaxmi Naresh Kalaskar

**Under the Guidance
of**

Mr. H. U. Joshi

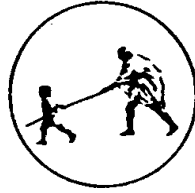
(Department of Computer Science and Engineering)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING
NANDED (M.S.)

Academic Year 2025-26

Certificate



This is to certify that the Internship entitled

“YouTube Channel Performance And Engagement Analytics Dashboard”

being submitted by Ms. Rajlaxmi Naresh Kalaskar to the Dr. Babasaheb Ambedkar Technological University, Lonere, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by her under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree.

Mr. H. U. Joshi
Project Guide

Dr. A. M. Rajurkar

H.O.D

Computer Science & Engineering

Dr. G. S. Lathkar

Director

MGM's College of Eng, Nanded

INTERNSHIP OFFER LETTER



Infosys Springboard Virtual Internship 6.0 | You have been shortlisted!

1 message

Infosys Springboard-Support <Springboard-support@infosys.com>

Thu, 15 Jan, 2026 at 4:57 pm



Dear Learner,

Congratulations! You have been selected for the [Infosys Springboard Internship 6.0 Batch 13](#).

What's next?

- Watch out for upcoming emails with more information

Got a query? Write to springboard-support@infosys.com

Regards,
Team Infosys Springboard

Scan the QR code to download
the Infosys Springboard App



App Store



Play Store

Follow us on [Facebook](#) | [Instagram](#) | [LinkedIn](#) | [X \(Twitter\)](#)



Important Updates: Infosys Virtual Internship6.0

1 message

Mentor 210 <springboardmentor210@gmail.com>
To: Mentor 210 <springboardmentor210@gmail.com>
Bcc: s22_kalaskar_rajaxmi@mgmcen.ac.in

Thu, 5 Feb, 2026 at 12:42 pm

Dear Interns,

I hope you're doing well. Below is a brief overview of your project details:

Project Title: YouTube Channel Performance and Engagement Analytics Dashboard

Technology Stack: Python

Mentor Name: Akhil

Coordinator Name: Mohammed Hussain

Call Timing: 6:45pm - 7:45pm

Call Link: <https://meet.google.com/yma-bhwh-efw>

You'll find the call schedule for the 8-week program attached along with the important Milestone Deadlines. Please note that attendance for the final evaluation presentation, which will take place after the 8-week program, is mandatory. Further details will be shared closer to the date.

You need to join the same meeting link for the calls on scheduled dates for our regular calls.

The link to the session recordings folder, where daily recordings will be uploaded, is

Link: https://drive.google.com/drive/folders/1cmBC8u9VsyaUKAIDhZP0PaqKUmsPMNGE?usp=drive_link

Lastly, please refer to the attached project document for further information.

Best Regards,
Mohammed Hussain

2 attachments

 **Call Schedule Interns.xlsx**
10 KB

 **YouTube Channel Performance and Engagement Analytics Dashboard.docx**
334 KB



Congratulations on successfully completing the Infosys Springboard Internship 6.0

1 message

Infosys Springboard-Support <Springboard-support@infosys.com>

Wed, 29 Apr, 2026 at 3:31 pm



Dear Intern,

Congratulations! You have successfully completed Infosys Springboard Internship 6.0! We're proud of your achievement and the dedication you've shown throughout the internship.

Your certificate will be sent to your registered email address shortly. Keep up the great work and continue striving for excellence!

Announce your achievement and about your internship experience on social media. Tag us @InfosysSpringboard when you do.

Note: Your project is your asset. Refrain from sharing your artifacts in online forums/domains to safeguard your work's ownership

Regards,
Team Infosys Springboard

Keep your learning journey going. Scan the QR code to download the **Infosys Springboard App** and access your courses on the go!



App Store



Play Store

Follow us on Facebook | Instagram | LinkedIn | X (Twitter) | Threads

In case of queries, write to springboard-support@infosys.com.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project guide, **Mr. H. U. Joshi** for his invaluable support, guidance, and encouragement throughout this project. His profound knowledge and expertise have been instrumental in the successful completion of this work. His patience and willingness to assist me at every step have greatly enriched my learning experience. His constructive feedback and insightful suggestions have not only helped me overcome challenges but also motivated me to strive for excellence.

I gladly take this opportunity to thank **Dr. A. M. Rajurkar** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded). We are heartily thankful to **Dr. G. S. Lathkar** (Director, MGM's College of Engineering, Nanded) for providing facilities during the progress of the project and also for her kind help, guidance and inspiration. Last but not least, we are also thankful to all those who helped, directly or indirectly, develop this project and complete it successfully.

With Deep Reverence,

Rajlaxmi Naresh Kalaskar [224]

[B.Tech CSE-B]

ABSTRACT

This report presents the development and implementation of the project titled “YouTube Channel Performance and Engagement Analytics Dashboard”, a data analytics system designed to collect, analyze, visualize, and compare YouTube channel performance data. With the rapid growth of digital content creation, YouTube creators need a simple and effective way to understand their channel growth, audience engagement, and overall performance. Manually checking different metrics such as subscribers, views, videos, likes, comments, and engagement can be time-consuming and difficult. This project addresses these challenges by providing an interactive dashboard that displays important channel insights in a structured and easy-to-understand format.

During the development of this project, I applied my academic knowledge in the fields of Python programming, API integration, database management, data visualization, and web application development. The system fetches public YouTube channel and video data using the YouTube Data API and stores the collected information in a MySQL database for further analysis. The dashboard calculates important metrics such as total subscribers, total views, total videos, average views, engagement rate, subscriber-to-view ratio, upload frequency, and performance share.

Throughout the project, I worked on backend processing, database connectivity, API handling, and frontend dashboard design using technologies such as Python, Streamlit, MySQL, Plotly, Pandas, YouTube Data API, and OpenRouter AI API. The system also includes a channel comparison module that allows users to compare multiple YouTube channels based on subscribers, views, videos, ranking, and benchmark analysis. Additionally, the project provides AI-generated growth strategies to help creators improve their channel performance.

Beyond technical implementation, this project provided practical exposure to real-world data analysis, API-based application development, visualization techniques, and AI-assisted recommendation systems. Overall, this project successfully provides a useful and interactive solution for analyzing YouTube channel performance and engagement, helping users make better decisions for content growth and improvement

TABLE OF CONTENTS

TOPICS	PAGE NO.
INTERNSHIP OFFER LETTER	I
ACKNOWLEDGEMENT	IV
ABSTRACT	V
TABLE OF CONTENTS	VI
LIST OF FIGURES	X
LIST OF TABLES	X
Chapter 1. INTRODUCTION	1
1.1 Introduction to Data Visualization Domain	1
1.2 Internship Objectives	2
1.3 Scope of the Project	3
1.4 Internship Duration and Deliverables	4
1.5 Introduction to YouTube Analytics Dashboard Systems	5
1.6 Problem Statement	5
1.7 Proposed Solution	6
1.8 Report Scope	7
Chapter 2. SYSTEM ARCHITECTURE	8
2.1 Architectural Design of the System	10
2.2 Major Components of the System	12
2.2.1 Core Processing Module	14
2.2.2 API Analysis and Recommendation Module	15
2.2.3 Validation and Metrics Module	16
2.3 Database Design	18
2.4 Data Flow and Module Communication	19
2.5 Technology Stack Integration	22

2.5.1 Python Backend	22
2.5.2 Streamlit Framework	22
2.5.3 Plotly Visualization	23
2.6 External Libraries and APIs	23
2.6.1 YouTube Data API	23
2.6.2 OpenRouter Integration	24
2.6.3 Pandas Library	24
2.6.4 SQLAlchemy and PyMySQL	25
2.6.5 Streamlit Standard Components	25
2.6.6 Scikit-learn and Requests	25
2.7 System Requirements	26
Chapter 3. FEATURE & FUNCTIONALITY	27
3.1 Data Collection	28
3.1.1 Channel Resolution using YouTube API	29
3.1.2 Video Metadata Collection	30
3.2 Channel Dashboard and Metric Generation	31
3.2.1 KPI Calculation	32
3.2.2 Category and Upload Feature Engineering	32
3.2.3 Paginated Video Table	33
3.3 Visualization and Engagement Analysis	33
3.3.1 Views and Upload Analysis	34
3.3.2 Engagement Analysis	34
3.3.3 Plotly Integration	35
3.4 Global Filter Feature	36
3.5 AI-Powered Assistance Features	37
3.5.1 Growth Strategy Insights	37
3.5.2 OpenRouter AI Recommendation	38

3.5.3 Local Recommendation Fallback	38
3.5.4 Topic Extraction	38
3.5.5 Performance Comparison	39
3.5.6 Help and FAQ	39
3.6 Technical Capabilities	40
3.6.1 Multiple Input Support	40
3.6.2 Batch API Processing	40
3.6.3 Duration and Date Conversion	41
3.6.4 Workspace Navigation	41
3.6.5 Environment Integration	41
3.6.6 Error Handling	42
Chapter 4. SYSTEM WORKFLOW & PROCESSING	43
4.1 Input File Handling Process	44
4.2 Source Data Parsing Sequence	45
4.3 Internal Data Processing	47
4.4 API Request and Response Cycle	48
4.5 Analysis Result Processing	49
4.6 Report Compilation Process	51
4.7 Data Storage and Retrieval	52
4.8 Execution Sequence of the System	53
4.9 Overall Processing Pipeline	54
Chapter 5. USER INTERFACE & EXPERIENCE	56
5.1 Data Collection Page	56
5.2 Channel Dashboard Interface	58
5.3 Visualization Interface	59
5.4 Channel Comparison Interface	60
5.5 Global Filter Interface	62

5.6 Growth Strategy Interface	63
5.7 Help and FAQ Interface	64
5.8 Dashboard Interface	65
CONCLUSION	67
REFERENCES	68

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
2.1	System Architecture	11
2.3	ER Diagram	18
2.4	Data Flow and Module Communication	20
4.5	Analysis Result Processing Workflow	50
5.1	Data Collection Page	57
5.2	Channel Dashboard Page	59
5.3	Visualization Page	60
5.4	Channel Comparison Page	61
5.5	Global Filter Sidebar	63
5.6	Growth Strategy Page	64
5.7	Help and FAQ Page	65
5.8	Complete Dashboard Interface	66

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.2	Major Components of the System	12
4.7	MySQL Table	53

INTRODUCTION

The Infosys Springboard Virtual Internship 6.0 is a learning-focused internship program created to help students gain practical experience in different technology domains through real-world projects and industry-based training. The program provides an opportunity to improve technical skills, analytical knowledge, and problem-solving abilities by working on hands-on development activities.

The internship in the Data Visualization domain helps students understand data analysis and visualization concepts using Python and related technologies. Participants work on practical projects that involve data collection, data cleaning, data processing, dashboard development, visual representation of data, and performance analysis techniques. This experience helps students connect academic learning with real development practices used in the software industry.

The program also supports skill development in areas such as logical thinking, project implementation, data interpretation, teamwork, and technical communication. By working on project-based tasks, students gain a better understanding of how data-driven systems are planned, developed, tested, and maintained in professional environments.

The YouTube Channel Performance and Engagement Analysis Dashboard project was developed during the Infosys Springboard Virtual Internship 6.0 under the Data Visualization domain. The project focuses on analyzing YouTube channel performance, audience engagement, video statistics, comment insights, and growth trends using data visualization techniques. It helps users understand channel growth, identify high-performing content, compare video metrics, and make data-driven decisions for improving YouTube channel performance.

1.1 Introduction to the Data Visualization Domain

Data Visualization is an important domain in data analysis that focuses on representing data in a visual and understandable format. It helps users analyze large amounts of information through charts, graphs, dashboards, and interactive reports. Data visualization is widely used in business analytics, social media analysis, marketing, education, finance, and decision-making systems. One of the main strengths of data

visualization is that it converts raw data into meaningful insights. Instead of showing only numbers or tables, visualization tools help users understand patterns, trends, comparisons, and relationships in data more easily. Technologies such as Python, Streamlit, Pandas, Plotly, Matplotlib, and YouTube Data API help in collecting, processing, analyzing, and presenting data in an interactive.

Data Visualization also supports better decision-making by making complex data simple and user-friendly. Interactive dashboards allow users to filter data, compare performance, identify growth trends, and understand engagement levels clearly. This makes it useful for both technical and non-technical users.

In this project, the Data Visualization domain is used to develop the YouTube Channel Performance and Engagement Analysis Dashboard. The project focuses on analyzing YouTube channel data such as views, likes, comments, subscribers, engagement rate, video performance, and growth trends. Visual dashboards are used to present this information through graphs, charts, and metrics, helping users understand channel performance and make data-driven decisions for improving content strategy.

1.2 Internship Objectives

The primary objective of the internship was to gain practical knowledge of data analysis and dashboard development by working on a real-world project in the Data Visualization domain. The internship helped in understanding how data-driven applications are developed, analyzed, and maintained using current technologies, programming practices, and visualization tools.

Another important goal was to study the role of data visualization in understanding YouTube channel performance, audience engagement, video trends, and content growth. The project provided hands-on experience in building an interactive dashboard that can reduce manual analysis and help users understand channel performance through charts, graphs, and visual reports.

The internship also focused on improving analytical skills, logical thinking, problem-solving abilities, and programming knowledge using Python and related technologies. Working on the project provided practical exposure to APIs, data collection, data cleaning, data processing, Streamlit-based dashboard development, and visualization libraries such as Pandas and Plotly.

In addition, the internship aimed to develop a better understanding of data interpretation, performance metrics, engagement analysis, and report generation. The

overall objective was to create a dashboard capable of analyzing YouTube channel data such as views, likes, comments, subscribers, video performance, engagement rate.

1.3 Scope of the Project

The YouTube Channel Performance and Engagement Analysis Dashboard project is designed to support data analysis, performance tracking, engagement measurement, and visual reporting for YouTube channels. The system aims to simplify the process of understanding channel growth and video performance by reducing the manual effort required to collect, process, and analyze YouTube data manually.

The project focuses on features such as fetching YouTube channel and video data, analyzing views, likes, comments, subscribers, engagement rate, subscriber-view ratio, and average views per video. It also helps users identify top-performing videos, low-performing videos, audience response, upload frequency, and content performance patterns. These insights allow users to understand which type of content is performing well and where improvement is required.

The application supports analysis using YouTube channel links, video links, or channel-related data, making it easier for users to access insights without manually searching for complex channel IDs. This improves usability for common users who may not have technical knowledge about YouTube APIs or channel identifiers. Interactive dashboards and visual reports are included to display performance metrics, engagement insights, comment analysis, and growth trends in a clear and user-friendly way.

The dashboard includes different visualization components such as bar charts, line charts, pie charts, metric cards, filters, and downloadable reports. These visual elements help users compare video performance, track growth over time, analyze engagement levels, and understand audience interaction more effectively. The system also supports data export options such as CSV, Excel, and PDF reports, which can be useful for documentation, presentation, and further analysis.

Technologies such as Python, Streamlit, Pandas, Plotly, YouTube Data API, and other data visualization libraries are used to build the system and support data collection, data processing, and visual analysis operations. The project is mainly developed for learning, data visualization, and social media analytics purposes.

It provides a strong foundation for future improvements such as AI-generated growth strategies, advanced sentiment analysis, improved comment analysis using

machine learning models, competitor comparison, real-time analytics, and more advanced YouTube performance prediction features. Overall, the project helps content creators, students, and analysts understand YouTube channel performance and make data-driven decisions.

1.4 Internship Duration and Deliverables

The Infosys Springboard Virtual Internship 6.0 was completed over a scheduled internship period focused on learning, project development, and practical implementation in the Data Visualization domain. During the internship, different phases of the project were carried out, including requirement understanding, system planning, data collection, dashboard development, testing, documentation, and final project preparation. The internship provided practical experience in working with real-world data analysis and visualization concepts. It helped improve technical, analytical, logical thinking, and problem-solving skills by working on a project based on YouTube channel data and performance analysis.

The major outcome of the internship was the development of the YouTube Channel Performance and Engagement Analysis Dashboard. The system includes features such as YouTube channel data fetching, video performance analysis, views, likes and comments analysis, engagement rate calculation, subscriber-view ratio, average views per video, comment analysis, growth trend visualization, and interactive dashboards for better understanding of channel performance.

The project also involved preparing reports, charts, graphs, dashboard screenshots, documentation, diagrams, testing outputs, and presentation materials related to the developed system. The dashboard provides visual insights through metric cards, bar charts, line charts, pie charts, filters, and downloadable reports such as CSV, Excel, and PDF.

The internship helped in gaining practical knowledge of data visualization, API integration, data cleaning, data processing, dashboard development, and performance analysis techniques. The completed project demonstrates how Python, Streamlit, Pandas, Plotly, and YouTube Data API can be combined to create an interactive and user-friendly dashboard that supports data-driven decision-making for YouTube channel growth and engagement improvement. It helped improve technical, analytical, logical thinking, and problem-solving skills by working on a project based on YouTube channel data and performance analysis.

1.5 Introduction to YouTube Analytics Dashboard Systems

YouTube Analytics Dashboard Systems are data visualization-based tools that help creators, analysts, students, and businesses understand the performance of YouTube channels and videos over time. These systems collect and analyze important metrics such as views, likes, comments, subscribers, upload frequency, engagement rate, average views per video, and subscriber-view ratio. The collected data is then presented through charts, graphs, metric cards, and interactive dashboards to support better decision-making. In traditional YouTube performance analysis, users need to manually check each video, compare views, likes, comments, publishing dates, titles, descriptions, and audience responses. This process becomes time-consuming and difficult when a channel contains many videos. A dashboard system reduces this manual effort by centralizing the data, processing it automatically, and converting it into meaningful visual insights.

The project supports individual channel analysis as well as multi-channel comparison. Individual channel analysis helps users study a selected channel in detail by examining video performance, engagement values, growth trends, and content patterns. Multi-channel comparison allows users to benchmark different channels side by side using subscribers, total views, uploaded videos, leaderboard ranking, and performance share.

The system also includes AI-assisted growth insights to make the dashboard more useful for content improvement. It identifies strong and weak content patterns based on high-performing and low-performing videos, engagement values, publishing behavior, and title patterns. Based on these observations, the system generates useful recommendations that can help users improve their content strategy, audience engagement, and channel growth.

The YouTube Channel Performance and Engagement Analysis Dashboard project is developed using these concepts to provide a simple, interactive, and user-friendly platform for YouTube data analysis. It combines data collection, data processing, visualization, comparison, and AI-based suggestions to help users understand channel performance and make data-driven decisions.

1.6 Problem Statement

YouTube creators, students, and analysts often need to understand why some videos perform better than others, which upload patterns are working, and how audience

engagement changes over time. However, YouTube data is usually spread across many pages such as channel pages, video pages, analytics sections, and comment sections. This makes it difficult to compare performance manually, especially when a channel contains many videos or when multiple channels need to be analyzed together.

Basic statistics such as views, likes, comments, and subscribers are easy to see on YouTube, but deeper analysis requires combining several values. For example, engagement rate depends on likes and comments in relation to views, while content strategy depends on comparing high-performing and low-performing videos across time, title patterns, upload frequency, and audience response. Manually calculating these metrics and identifying useful patterns can be time-consuming and may lead to inaccurate conclusions.

Another major problem is the absence of a simple educational tool that combines YouTube data collection, database storage, filters, visual analysis, channel comparison, comment analysis, and growth suggestions in one place. In many cases, users need to use separate tools for API data collection, spreadsheets, chart creation, manual calculations, and note-making. This increases effort, makes the workflow less organized, and becomes difficult for non-technical users to manage.

To overcome these issues, the proposed YouTube Channel Performance and Engagement Analysis Dashboard is developed as a single platform where users can collect YouTube data, store it, filter it, visualize it, compare channels, and review actionable growth recommendations. The dashboard helps users analyze video performance, audience engagement, upload trends, and channel growth through interactive charts and metrics. By combining data visualization and AI-assisted insights, the project reduces manual effort and supports better data-driven decisions for improving YouTube content strategy.

1.7 Proposed Solution

The proposed solution for this project is a YouTube Channel Performance and Engagement Analysis Dashboard that helps users collect, analyze, visualize, and compare YouTube channel data in an interactive way. The system is developed using Python, Streamlit, YouTube Data API, MySQL, Pandas, Plotly, and OpenRouter integration. It provides a single platform where users can view channel performance, video statistics, engagement insights, and growth recommendations. The application collects public YouTube channel and video data and stores it in a database for further

analysis. The data collection module accepts flexible YouTube inputs such as channel ID, channel handle, channel URL, user URL, custom link, and video link. These inputs are first converted into a valid channel ID, and then the system fetches channel details, video information, and performance metrics using the YouTube Data API.

The dashboard module displays important information such as channel profile, total subscribers, total views, total videos, views, likes, comments, engagement rate, average views per video, and subscriber-view ratio. It also includes paginated video tables and video URLs so that users can easily review individual video performance. The visualization module presents data through charts, graphs, metric cards, and filters to show trends, engagement distribution, upload behavior, and content performance.

The system also includes a comparison module that allows users to compare up to five YouTube channels. This feature helps users analyze leaderboard ranking, benchmark performance, subscribers, views, uploaded videos, and performance share. It is useful for understanding how one channel performs compared to others.

The growth strategy module adds intelligence to the project by identifying strong and weak videos and generating useful recommendations. The application uses OpenRouter AI if configured, and if the AI service is unavailable, it falls back to local rule-based strategy generation. By combining data visualization, comparison, database storage, and AI-assisted insights, the project provides an effective solution for YouTube channel performance analysis and data-driven content improvement. The visualization module presents data through charts, graphs, metric cards, and filters to show trends, engagement distribution, upload behavior, and content performance.

1.8 Report Scope

This report presents the design, implementation, workflow, modules, interfaces, and outcomes of the YouTube Channel Performance and Engagement Analysis Dashboard developed during the internship project. The main purpose of the system is to simplify YouTube channel analysis by collecting public YouTube data, storing it in MySQL, processing it with Python, and displaying meaningful analytics through Streamlit and Plotly.[\[1\]](#) The report explains the architecture of the application, the role of different modules, and the technologies used during development. It describes how features such as YouTube data collection, channel ID resolution, database storage, data cleaning, data processing, metric calculation, dashboard visualization, channel comparison, and AI-assisted growth strategy generation are implemented within the system.

SYSTEM ARCHITECTURE

The YouTube Channel Performance and Engagement Analysis Dashboard is designed using a modular and well-structured architecture that allows the system to perform YouTube data collection, processing, analysis, visualization, comparison, and growth recommendation tasks efficiently. The architecture is organized in such a way that each component handles a separate responsibility, including API communication, input resolution, database operations, metric calculation, filtering, visualization, channel comparison, and growth strategy generation. This modular design improves the flexibility, maintainability, and scalability of the application, making it easier to upgrade in the future with features such as sentiment analysis, scheduled data refresh, predictive analytics, and advanced AI-based recommendations.

The complete workflow of the system begins from the Streamlit user interface layer, where users interact with the application through a web-based dashboard. Users can navigate through different pages such as Data Collection, Channel Dashboard, Visualizations, Channel Comparison, Growth Strategy, and Help and FAQ. The interface is designed to be simple, interactive, and user-friendly so that students, creators, analysts, and non-technical users can use the system without difficulty. The sidebar also provides global filters such as date range, metric selection, and channel/video-based filtering, which affect dashboard and visualization results.

The data collection module accepts different types of YouTube inputs such as channel ID, channel handle, channel URL, custom URL, user URL, and video link. These inputs are first processed by the input resolution module, which converts them into a valid YouTube channel ID. After resolving the channel ID, the system communicates with the YouTube Data API to fetch public channel information and video details such as video title, publish date, views, likes, comments, duration, description, and video URL.

The backend processing layer forms the core part of the system architecture. It includes the YouTube API module, database operation module, metric calculation module, and filter module. These components communicate with the external YouTube Data API and the local MySQL database to support reliable data collection, storage, and

retrieval. The fetched data is cleaned and processed using Python libraries such as Pandas, and then stored.

After the data is collected and stored, the metric calculation module calculates important performance indicators such as total views, total likes, total comments, subscribers, average views per video, engagement rate, subscriber-view ratio, upload frequency, and top-performing videos. These metrics help users understand how the channel is performing and how audiences are interacting with the content. The filter module allows users to refine results based on selected conditions, making the analysis more focused and meaningful.

The results generated by the backend modules are then transferred to the visualization and reporting layer. This layer uses Streamlit and Plotly components to display metric cards, bar charts, line charts, pie charts, tables, and interactive visual reports. The dashboard presents channel profile information, key metrics, advanced metrics, paginated video tables, video URLs, engagement analysis, upload behavior, and growth trends in a clear visual format. These visualizations help users easily identify patterns, compare performance, and make data-driven decisions.[\[2\]](#)

The system also includes a channel comparison module that allows users to compare up to five YouTube channels at the same time. This module provides leaderboard ranking, benchmark analysis, subscriber comparison, total views comparison, uploaded video comparison, and performance share analysis. It presents the comparison results in a clear and visual format, making it easier for users to identify the strongest and weakest performing channels. By comparing important metrics side by side, users can understand how one channel performs against other selected channels and can use these insight.

The growth strategy module adds intelligence to the project by identifying strong and weak videos and generating recommendations for improvement. The application uses OpenRouter AI if configured, and if the AI service is unavailable, it falls back to local rule-based strategy generation. This ensures that users can still receive useful suggestions even when the AI service is not active.

The architecture of the system ensures smooth communication between all components, including the frontend interface, YouTube API module, database layer, processing modules, visualization components, comparison module, and AI-based recommendation system. Overall, the architecture of the YouTube Channel

Performance and Engagement Analysis Dashboard provides a reliable and efficient framework for YouTube data analysis, performance tracking, engagement measurement, visualization, and content growth improvement.

2.1 Architectural Design of the System

The architectural design of the YouTube Channel Performance and Engagement Analysis Dashboard is developed to provide an organized, modular, and efficient environment for YouTube data collection, processing, visualization, comparison, and growth analysis. The system is structured into multiple layers where each component performs a specific task while interacting with other modules to complete the overall workflow. This design approach improves the maintainability, scalability, and flexibility of the application, allowing new features and technologies to be added easily in the future.

The architecture begins with the user interface layer, which is developed using Streamlit. This layer acts as the main interaction point between the user and the system. Through the web interface, users can navigate different pages such as Data Collection, Channel Dashboard, Visualizations, Channel Comparison, Growth Strategy, and Help and FAQ. The interface is designed to be simple, interactive, and user-friendly so that students, creators, analysts, and non-technical users can use the application without difficulty.

Once the user enters a YouTube channel ID, handle, channel URL, or video link, the input is passed to the input resolution module. This module converts different types of YouTube inputs into a valid channel ID. After the channel ID is resolved, the system communicates with the YouTube Data API to fetch public channel information and video details such as channel name, subscribers, total views, video count, video title, publish date, views, likes, comments, duration, description, and video URL. The collected data is then forwarded to the database and backend processing layer. In this layer, the data is stored in a MySQL database for future use and further analysis. The backend also includes data cleaning, filtering, and metric calculation using Python and Pandas.

The system also contains a global filter module that allows users to refine the analysis based on selected conditions such as date range, metrics, channel data, or video data. These filters help users focus on specific time periods or performance values, making the analysis more accurate and meaningful.

This layer uses Streamlit and Plotly to present charts, graphs, metric cards, paginated video tables, and interactive visual reports. These visualizations help users understand channel performance, audience engagement, upload behavior, video trends, and content performance in a clear graphical manner. The architecture also includes a channel comparison module, which allows users to compare up to five YouTube channels. This module provides leaderboard ranking, benchmark analysis, subscriber comparison, views comparison, uploaded video comparison, and performance share analysis. It helps users understand how one channel performs compared to other selected channels.

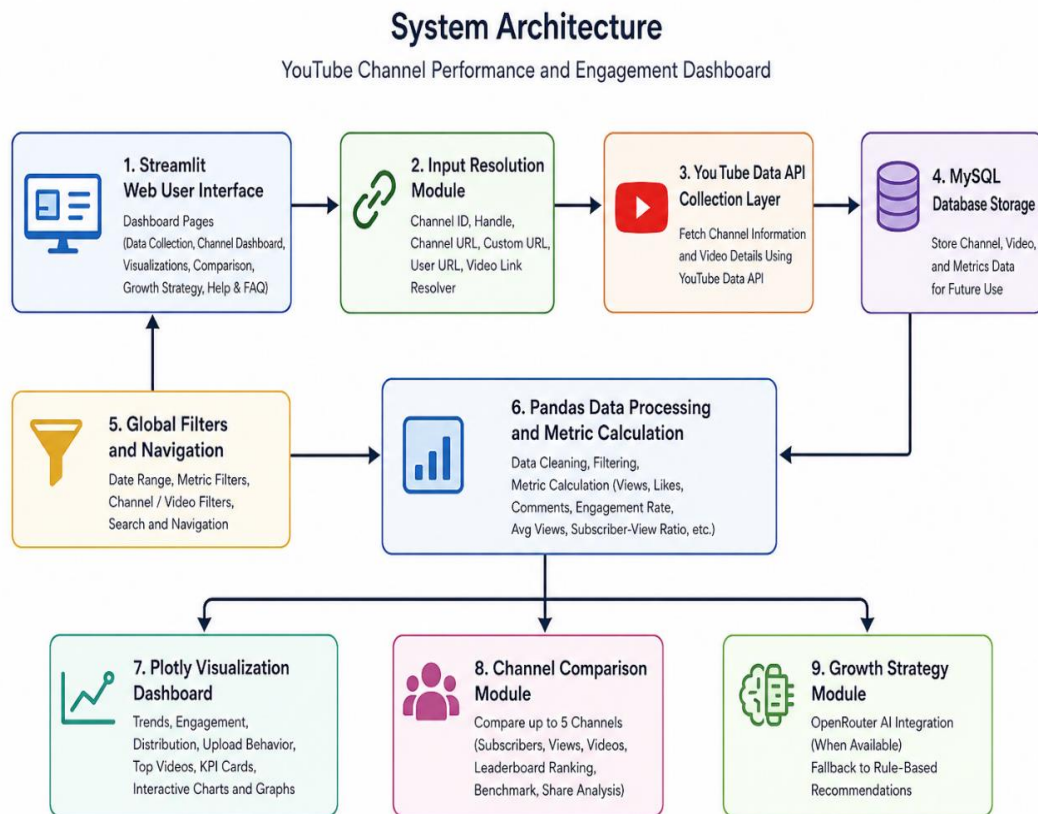


Figure 2.1: System Architecture

The growth strategy module adds intelligence to the system by identifying strong and weak videos and generating improvement recommendations. The application uses OpenRouter AI integration when available, and if the AI service is unavailable, it uses local rule-based logic to generate useful growth suggestions.

The system also includes a Help and FAQ section to guide users in understanding how to use the dashboard effectively. This section provides basic instructions for data collection, channel link or video link input, dashboard navigation,

filter usage, visualization interpretation, channel comparison, and growth strategy features. It helps beginners and non-technical users understand the workflow of the application and solve common doubts while using the system.

Overall, the architecture of the proposed system provides a smooth workflow for YouTube data collection, database storage, data processing, filtering, visualization, channel comparison, and growth recommendation generation. This modular architecture makes the system reliable, easy to maintain, and suitable for future enhancements such as sentiment analysis, scheduled data refresh, predictive growth modeling, and stronger personalized strategy suggestions.

2.2 Major Components of the System

The YouTube Channel Performance and Engagement Analysis Dashboard is designed using multiple interconnected components, where each component handles a specific task within the system. Together, these components manage operations such as YouTube data collection, input resolution, database storage, metric calculation, dashboard display, visualization, channel comparison, growth strategy generation, filtering, and user guidance. The modular design keeps the application organized, makes maintenance easier.

Component	Purpose
Streamlit application	Provides navigation, page layout, sidebar filters, and user interaction.
YouTube API module	Resolves inputs and fetches channel, video, search, and comment data.
Database module	Stores and retrieves channel and video records from MySQL.
Dashboard module	Displays channel profile, KPI cards, advanced metrics, and video table.
Visualization module	Creates Plotly charts for trends, distribution, funnel, and heatmap analysis.

Comparison module	Compares up to five channels using leaderboard, benchmarks, and performance share.
Growth strategy module	Generates AI or local recommendations based on video performance.

Table 2.2: Major Components of the System

One of the main components of the system is the Streamlit Application Shell. It provides the main user interface, page layout, sidebar navigation, and user interaction. Through this interface, users can access pages such as Data Collection, Channel Dashboard, Visualizations, Channel Comparison, Growth Strategy, and Help and FAQ. The sidebar also provides global filters that help users refine the displayed results.

The YouTube API Module is responsible for resolving different types of YouTube inputs and fetching required data. It accepts channel IDs, handles, channel URLs, custom URLs, user URLs, and video links. After converting the input into a valid channel ID, it collects channel information, video details, search data, and comment-related data using the YouTube Data API.[\[3\]](#)

The Database Operation Module manages the storage and retrieval of data using MySQL. It stores channel records, video records, metrics, and related information so that the application can reuse previously collected data without fetching everything again. This improves efficiency and supports organized data management. The Dashboard Module displays important channel information in a clear and structured format. It shows the channel profile, KPI cards, advanced metrics, paginated video tables, and video URLs. This helps users quickly understand the overall performance of a selected YouTube channel.

The Visualization Module creates interactive charts and graphs using Plotly. It presents trends, distribution analysis, engagement patterns, upload behavior, funnel analysis, and heatmap analysis. These visualizations make complex YouTube data easier to understand and support better decision-making.

The Comparison Module allows users to compare up to five YouTube channels. It provides leaderboard ranking, benchmark comparison, subscriber comparison, views comparison, uploaded video comparison, and performance share analysis. This helps users understand how one channel performs compared to others.

The Growth Strategy Module generates improvement suggestions based on video performance. It identifies strong and weak videos, engagement patterns, title patterns, and upload behavior. The module uses OpenRouter AI when available and falls back to local rule-based recommendations when the AI service is unavailable. The system also includes a Filter Utility Module that applies global filters such as date range, metric selection, channel filters, and video filters. These filters help users focus on specific data and generate more meaningful insights. Finally, the Help and FAQ Page provides guidance for users. It explains how to collect data, use channel or video links, navigate the dashboard, apply.

2.2.1 Core Processing Module

The Core Processing Module is one of the most important components of the YouTube Channel Performance and Engagement Analysis Dashboard. This module is responsible for preparing YouTube channel and video data for analytics. It acts as the central processing unit of the application and provides the foundation for features such as dashboard analysis, metric calculation, visualization, channel comparison, and growth strategy generation. The processing workflow begins when the user enters a YouTube input through the Streamlit interface. The input may be a channel ID, channel handle, channel URL, custom URL, user URL, or video link. The core processing module receives this input and resolves it into a valid YouTube channel ID. This step is important because the YouTube Data API requires a proper channel ID to fetch accurate channel and video information.

After resolving the channel ID, the module communicates with the YouTube Data API to collect public channel details and video data. It fetches information such as channel name, subscribers, total views, total videos, video title, publish date, thumbnail, duration, views, likes, comments, description, and video URL. The module also handles pagination through upload playlists so that multiple videos from a channel can be collected properly. Once the data is fetched from the API, the module converts the raw API responses into structured records and dataframes using Pandas. During this stage, it performs important processing tasks such as date conversion, duration conversion, thumbnail selection, missing value handling, and formatting of statistics. These steps make the raw YouTube data clean, organized, and suitable for dashboard analysis. It explains how to collect data, use channel or video links, navigate the dashboard, apply.

The processed data is then passed to the database functions, where channel and video records are stored in MySQL for future use. The same structured data is also used by other modules such as the dashboard module, visualization module, comparison module, filter utility, and growth strategy module. Overall, the Core Processing Module plays a key role in converting raw YouTube API data into meaningful and usable information. It ensures that the collected data is accurate, structured, and ready for analysis, helping the system display reliable metrics, charts, comparisons, and recommendations.

2.2.2 API Analysis and Recommendation Module

The API Analysis and Recommendation Module is responsible for analyzing stored YouTube video data and generating useful growth suggestions for the channel. This module plays an important role in helping users understand which videos are performing well, which videos need improvement, and what content strategies can be followed to improve

audience engagement and channel growth. One of the major functions of this module is video performance analysis. The module studies stored video data from the database and separates high-performing and low-performing videos using quantile-based thresholds. It compares videos based on views, likes, comments, engagement rate, publishing date, and other performance values. This helps the system identify strong videos, weak videos, and overall content performance patterns.

The module also calculates important metrics such as average views, total comments, engagement rate, upload frequency, and best publishing patterns. It also studies topic signals from video titles to understand what type of content is performing better. These calculated insights are useful for generating recommendations related to content planning, title improvement, posting schedule, and audience engagement.

When OpenRouter AI is configured, the module prepares a compact evidence pack containing important channel statistics, high-performing videos, low-performing videos, engagement values, and content patterns. This evidence pack is sent to the selected AI model, which generates channel-specific growth advice. The recommendations may include suggestions for improving titles, posting consistency, content topics, audience interaction, and overall video strategy. If the OpenRouter API is not available or not configured, the system does not stop working. Instead, the module The module also calculates important metrics such as average views, total comments,

engagement rate, upload frequency uses local fallback rules to generate useful recommendations. These rule-based suggestions are created from calculated metrics and video performance patterns.

The API Analysis and Recommendation Module adds intelligence to the dashboard by combining YouTube data analysis, performance comparison, AI-based suggestions, and local rule-based recommendations. It helps users make better data-driven decisions for improving YouTube channel performance and engagement.

2.2.3 Validation and Metrics Module

The Validation and Metrics Module is developed to check whether the collected YouTube channel and video data is suitable for analysis. Its main purpose is to handle data quality issues, calculate important performance metrics, and ensure that the dashboard displays accurate and meaningful results. By automating these checks, the module reduces manual calculation work and helps users understand channel performance more effectively.

One of the key tasks performed by this module is data validation. The system checks whether the available channel and video data is complete and usable. It handles situations such as empty dataframes, missing values, unavailable comments, missing likes, and zero-view cases. These checks are important because YouTube data may sometimes be incomplete or unavailable due to privacy settings, disabled comments, or API limitations.

The module also performs metric calculation for channel and video analysis. It calculates important values such as total views, total likes, total comments, average views per video, engagement rate, subscriber-view ratio, upload frequency, and top-performing videos. These metrics help users understand how well the channel is performing and how audiences are interacting with the content.

Engagement rate is calculated safely by using views, likes, and comments. In cases where a video has zero views, the system avoids calculation errors by replacing zero values where needed. This prevents division errors and ensures that the dashboard continues to work smoothly even when some videos have very low or missing performance data.

The module also works with global filters to provide focused analysis. Filters such as date range, metric selection, channel filters, and video filters are applied before

displaying results. This allows users to analyze specific time periods, selected videos, or particular performance metrics more clearly. The Validation and Metrics Module improves the reliability of the dashboard by cleaning data, handling missing or invalid values, and calculating accurate performance indicators. It ensures that the YouTube Channel Performance and Engagement Analysis Dashboard provides useful insights for performance tracking, visualization, comparison, and growth strategy generation.

2.2.4 Reporting and Storage Module

The Reporting and Storage Module handles the storage, presentation, and review of all YouTube channel and video analysis data produced by the system. This module plays an important role in organizing collected data in a structured format and displaying it in a clear and understandable way. It helps users view channel performance, video statistics, comparison results, and growth recommendations without directly checking raw database records. One of the main tasks performed by this module is storing data in MySQL tables. The system stores channel records such as channel ID, channel name, profile details, subscriber count, total views, total videos, thumbnails, and analysis timestamps. Similarly, video records include video ID, title, description, publishing date, duration, views, likes, comments, thumbnails, and video URLs. This structured storage allows the application to retrieve previously collected data whenever required.

The reporting part of the application displays the stored data through metric cards, paginated tables, charts, leaderboards, and recommendation panels. Metric cards show key values such as subscribers, total views, total videos, average views, engagement rate, and subscriber-view ratio. Paginated tables help users review video-level details in an organized way, while charts and graphs make trends, engagement patterns, and upload behavior easier to understand.

The module also supports channel comparison reporting. It presents leaderboard rankings, benchmark results, subscriber comparison, views comparison, uploaded video comparison, and performance share analysis. These reports help users understand how one channel performs compared to other selected channels.

In addition, the module displays growth recommendations generated by the Growth Strategy Module. These recommendations are shown in a simple panel format so users can easily understand improvement areas such as content topics, title patterns, upload consistency, and engagement improvement. The Reporting and Storage Module

improves the usability of the dashboard by storing YouTube data properly and presenting it in a visual and user-friendly format. It avoids dependence on raw database inspection and allows users to review insights directly through the dashboard.

2.3 Database Design

The Database Design of the YouTube Channel Performance and Engagement Analysis Dashboard is created to store YouTube channel data, video details, comments, metrics, comparison results, and growth strategy recommendations in an organized manner. The system uses MySQL as the database because it provides structured storage, easy data retrieval, and reliable management of large records.

The database acts as the central storage layer of the project. After the user enters a YouTube channel link, video link, handle, or channel ID, the system fetches data using the YouTube Data API and stores it in database tables. This helps avoid repeated API calls and allows the dashboard to reuse previously collected data for analysis, visualization, comparison, and recommendation generation.

The following ER diagram shows the relationship between different database tables such as channels, videos, comments, metrics, growth strategies, and comparison records.

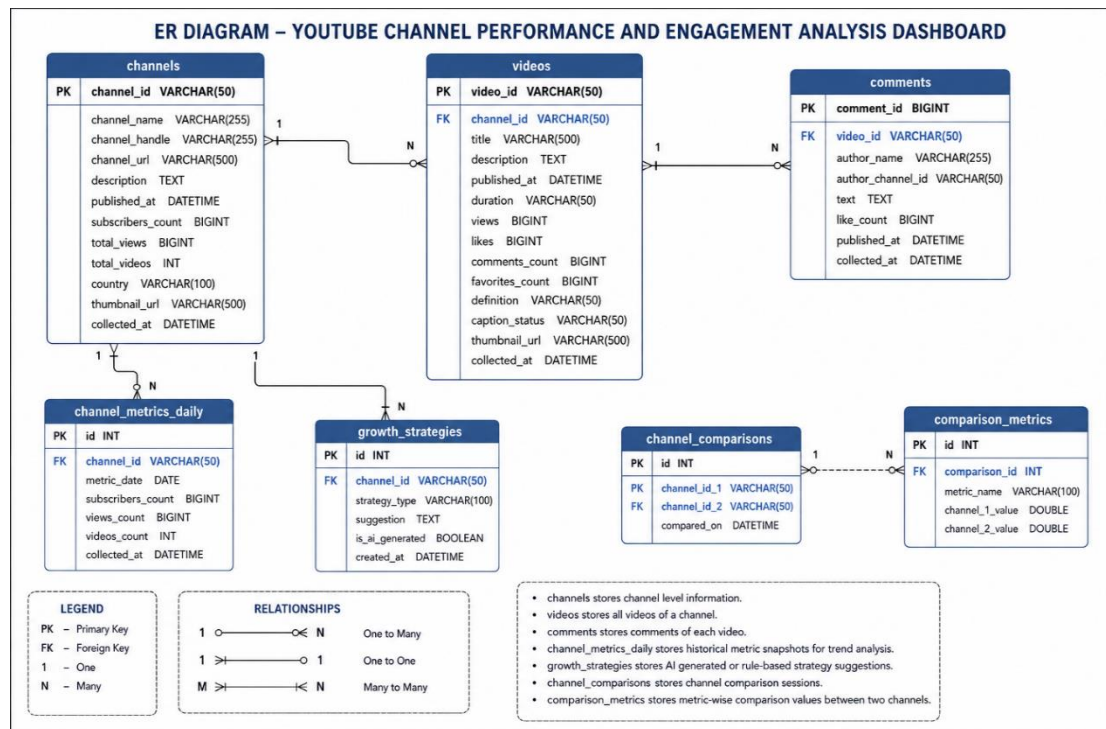


Figure 2.3: ER Diagram

The database acts as the central storage layer of the project. After the user enters a YouTube channel link, video link, handle, or channel ID, the system fetches data using the YouTube Data API and stores it in database tables. This helps avoid repeated API calls and allows the dashboard to reuse previously collected data for analysis, visualization, comparison, and recommendation generation.

The main tables used in the database are channels, videos, comments, channel metrics, growth strategies, channel comparisons, and comparison metrics. Each table stores a specific type of information. The channels table stores channel-level details, while the videos table stores all video-related data of a channel. The comments table stores selected comment data from videos. The metrics and strategy tables store calculated values and recommendation results.

The database follows a relational structure. One channel can have many videos, so the relationship between the channels table and videos table is one-to-many. One video can have many comments, so the relationship between videos and comments is also one-to-many. Similarly, one channel can have many metric records and many growth strategy records. The comparison tables are used to store channel comparison results and metric-wise comparison values. The ER Diagram represents this database structure visually. It shows the primary keys, foreign keys, table fields, and relationship between different.

2.4 Data Flow and Module Communication

The Data Flow and Module Communication process explains how different modules of the YouTube Channel Performance and Engagement Analysis Dashboard exchange information during data collection, storage, processing, visualization, comparison, and growth recommendation generation. The system follows a structured workflow where the output of one module becomes the input for another module. This organized communication between components ensures smooth execution and improves the efficiency of the application.

The process starts when the user enters a YouTube input on the Data Collection page through the Streamlit-based web interface. The input can be a channel ID, channel handle, channel URL, custom link, user URL, or video link. The Input Resolver Module checks the input type and converts it into a valid YouTube channel ID. This step is important because the YouTube Data API requires a proper channel ID to fetch accurate channel and video information.[\[4\]](#)

After the channel ID is available, the system communicates with the YouTube Data API to fetch channel details such as channel name, subscribers, total views, total videos, profile image, and description. The system also identifies the channel's uploads playlist. From this playlist, video IDs are collected, and video details are fetched in batches. These video details include video title, publish date, duration, views, likes, comments, description, thumbnail, and video URL.

Once the data is collected, the results are passed to the Database Module, where channel and video records are inserted or updated in the MySQL database. Storing data in the database helps avoid repeated API calls for every dashboard display. It also allows users to analyze previously collected channels even after the data collection process has finished.

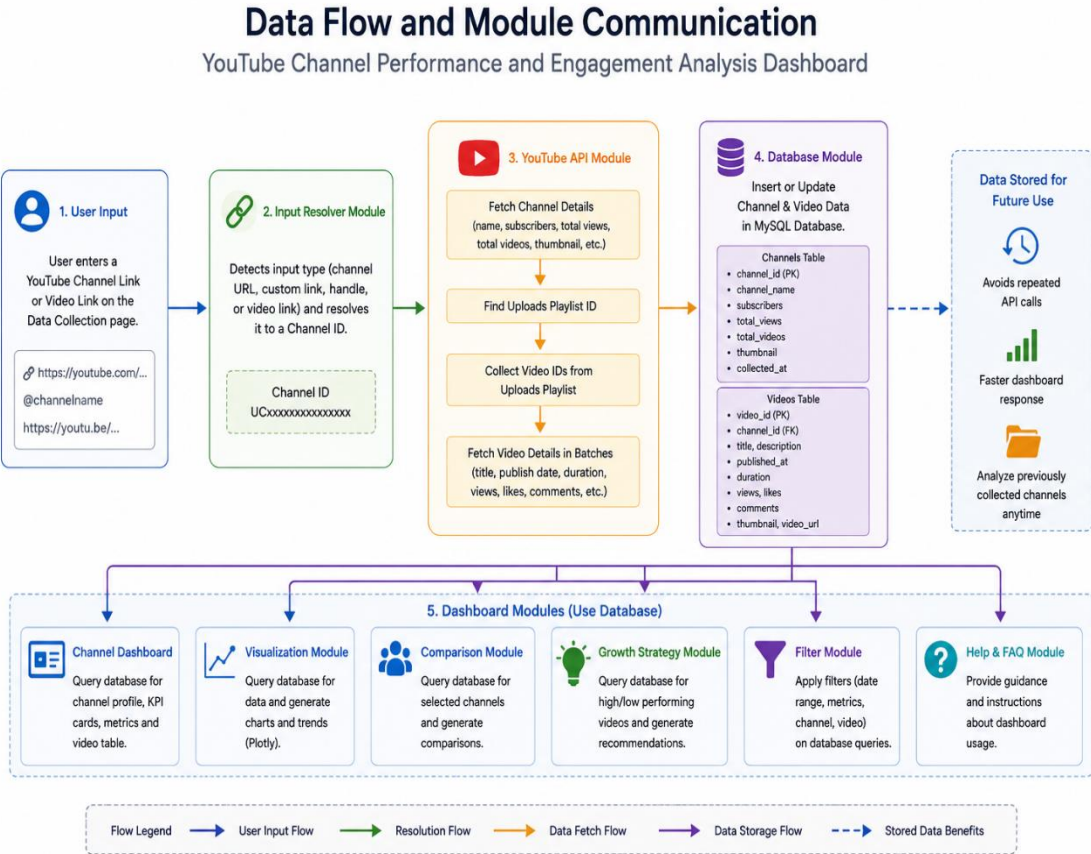


Figure 2.4: Data Flow and Module Communication

Once the data is collected, the results are passed to the Database Module, where channel and video records are inserted or updated in the MySQL database. Storing data in the database helps avoid repeated API calls for every dashboard display. To analyze previously collected channels even after the data collection process has finished.

The Dashboard, Visualization, and Comparison pages query the database instead of calling the API every time. This improves responsiveness and makes the application faster and more reliable. The retrieved data is processed using Pandas, where cleaning, formatting, date conversion, duration conversion, missing value handling, and metric calculations are performed. Metrics such as engagement rate, average views per video, subscriber-view ratio, total likes, total comments, upload frequency, and top-performing videos are calculated from the stored data.

The processed data is then passed to the Visualization Module, where Plotly charts, graphs, metric cards, and tables are generated. The dashboard displays channel profile information, KPI cards, advanced metrics, paginated video tables, engagement charts, upload trends, distribution charts, and other visual insights. The Global Filter Module communicates with these pages to apply filters such as date range, metric selection, channel selection, and video-level filtering.

For multi-channel analysis, the Comparison Module retrieves selected channel records from the database and compares up to five channels. It generates leaderboard rankings, benchmark charts, subscriber comparison, views comparison, uploaded video comparison, and performance share analysis. This helps users understand how one channel performs compared to other channels.

The Growth Strategy Module uses the stored and processed video data to identify high-performing and low-performing videos. It calculates performance patterns, engagement values, title signals, and upload behavior. If OpenRouter AI is configured, the module sends a compact evidence pack to the AI model and receives channel-specific growth advice. If the AI service is unavailable, local rule-based recommendations are generated without stopping the dashboard.

Finally, all results are displayed through the Streamlit interface in a clear and interactive format. The modular communication structure ensures efficient data transfer between the user interface, input resolver, YouTube API module, database module, processing logic, filters, visualization pages, comparison module, and recommendation module.

Overall, this workflow provides a reliable and organized system for YouTube data analysis, performance tracking, engagement measurement, channel comparison, and growth improvement. If the AI service is unavailable, local rule-based recommendations are generated without stopping the dashboard.

2.5 Technology Stack Integration

The YouTube Channel Performance and Engagement Analysis Dashboard is developed using multiple technologies and software tools that work together to provide data collection, storage, processing, visualization, comparison, and growth recommendation features. The integration of these technologies helps the system work efficiently while maintaining flexibility and scalability for future improvements.

Python acts as the main programming language for backend logic. Streamlit provides the web-based user interface. Pandas handles dataframes, cleaning, and transformations. Plotly creates interactive charts and graphs. MySQL stores channel and video data, while SQLAlchemy manages database communication. Together, these technologies create a complete analytics application for YouTube performance analysis.

2.5.1 Python Backend

The backend of the YouTube Channel Performance and Engagement Analysis Dashboard is developed using Python. It contains all major business logic of the system, including YouTube API calls, input resolution, API response processing, database insertions, metric calculations, filters, visualization data preparation, channel comparison, and recommendation generation.

Python was selected because it is simple, flexible, and widely used for data analysis, API integration, and dashboard development. It supports many useful libraries that make data handling and visualization easier. In this project, Python helps process YouTube channel and video data, calculate important metrics such as engagement rate, average views

Per video, subscribe-view ratio, and upload frequency, prepare the data for dashboard. The modular structure of the Python backend makes the project easier to maintain and extend. New features such as sentiment analysis, scheduled data refresh, predictive growth modeling, or advanced AI-based recommendations can be added in the future without changing the entire system.

2.5.2 Streamlit Framework

The user interface of the YouTube Channel Performance and Engagement Analysis Dashboard is developed using the Streamlit framework. Streamlit is a Python-based web framework used for creating interactive dashboards and data-driven applications with less frontend development effort. In this project, Streamlit provides pages, sidebar controls, text inputs, buttons, dataframes, tabs, expanders, progress bars, metric panels,

and chart rendering. Users can navigate through pages such as Data Collection, Channel Dashboard, Visualizations, Channel Comparison, Growth Strategy, and Help and FAQ. The sidebar also provides global filters for date range, metrics, channel selection, and video-level filtering.

The dashboard uses Streamlit session state to preserve filters and page-specific values during user interaction. This improves the user experience because selected filters and inputs remain available while navigating between pages. Streamlit also integrates directly with Python backend modules, allowing smooth communication between the interface, data processing logic, and visualization components.

2.5.3 Plotly Visualization

The YouTube Channel Performance and Engagement Analysis Dashboard uses Plotly to generate interactive charts, graphs, and visual reports. Plotly helps present YouTube data in a clear and easy-to-understand format instead of showing only raw numbers or tables.

In this project, Plotly is used for interactive visualizations such as line charts, bar charts, scatter plots, histograms, heatmaps, funnel charts, and pie charts. These charts help users analyze trends, engagement distribution, upload behavior, video performance, and channel comparison results.

Plotly supports hover details and responsive rendering, making it suitable for exploring YouTube performance data interactively. Users can view detailed values by hovering over charts and understand patterns more easily.

2.6 External Libraries and APIs

The YouTube Channel Performance and Engagement Analysis Dashboard uses several external libraries and APIs to support data collection, processing, storage, visualization, and recommendation generation. These libraries help the system work efficiently and reduce the complexity of development. Each library performs a specific role in the project and supports smooth communication between different modules.

2.6.1 YouTube Data API

The YouTube Data API is one of the most important external services used in this project. It is used to fetch public YouTube channel and video information required for analysis. The application uses API endpoints such as channels, search, playlistItems, videos, and commentThreads to collect data. Through this API, the system retrieves details such as channel name, channel ID, subscriber count, total views, total videos,

thumbnails, video titles, publish dates, views, likes, comments, duration, and video URLs. This collected data becomes the base for dashboard metrics, visualizations, comparison, and growth strategy generation. The API also helps the system resolve different types of user inputs, such as channel links, handles, custom URLs, and video links. By using the YouTube Data API, the dashboard can collect accurate and updated data directly from YouTube, which improves the reliability of analysis.

2.6.2 OpenRouter Integration

OpenRouter is used in the Growth Strategy Module to generate AI-based recommendations for YouTube channel improvement. The system reads OpenRouter configuration from environment variables, such as API key and selected model name.

When OpenRouter is available, the system sends important channel performance details, high-performing videos, low-performing videos, engagement values, and title patterns to the AI model. Based on this evidence, the AI generates useful growth suggestions. If OpenRouter is not configured or unavailable, the system uses free-model fallback options or local rule-based recommendations, making the feature flexible and reliable.

This integration adds intelligence to the dashboard by helping users understand what actions can improve their channel performance. It supports suggestions related to content topics, upload consistency, title improvement, engagement improvement.

2.6.3 Pandas Library

Pandas is used for data processing and transformation in the project. It supports dataframe creation, sorting, grouping, filtering, date conversion, aggregation, quantile calculation, and derived metric creation.

In this dashboard, Pandas converts raw YouTube API responses and database records into analytics-ready data. It helps calculate metrics such as engagement rate, average views per video, total likes, total comments, subscriber-view ratio, upload frequency, and high-performing or low-performing video groups. Pandas plays a central role in preparing clean and structured data for charts and reports. Pandas also helps handle missing values, format dates, convert numeric columns, and apply global filters. This makes the data more accurate and suitable for visualization. Without Pandas, processing large video datasets and calculating performance metrics would be more difficult and time-consuming. This integration adds intelligence to the dashboard by helping users understand what actions can improve their channel performance.

2.6.4 SQLAlchemy and PyMySQL

SQLAlchemy and PyMySQL are used for database connectivity between the Python application and the MySQL database. SQLAlchemy helps manage database communication in a structured way, while PyMySQL acts as the connector for MySQL.

The project uses connection pooling, pre-ping checks, parameterized SQL queries, and transaction blocks to make database operations more reliable and secure. These libraries help insert, update, retrieve, and manage channel and video records efficiently. They also reduce repeated API calls by allowing previously collected data to be reused from the database. Using these libraries improves the stability of database operations and helps maintain organized data storage. They also make it easier to perform queries for dashboard display, visualization, comparison, and growth strategy generation.

2.6.5 Streamlit Standard Components

Streamlit standard components are used to build the interactive web interface of the dashboard. Components such as select boxes, multiselects, radio buttons, date inputs, tabs, expanders, progress bars, buttons, metric cards, and dataframes are used throughout the application.

These components help users enter YouTube links, collect data, apply filters, view charts, compare channels, and access growth strategies. Streamlit makes the dashboard easy to use and allows users to interact with data without needing technical knowledge of APIs or databases. The components also improve user experience by making the application simple and interactive. For example, progress bars show the status of data collection, metric cards display important values quickly, and tabs organize information into readable sections. Using these libraries improves the stability of database operations and helps maintain organized data storage.

2.6.6 Scikit-learn and Requests

Scikit-learn is used in the project for TF-IDF based phrase extraction during title analysis. This helps identify important words or topic signals from video titles. These extracted phrases are useful for understanding what type of content performs better on a channel.

The Requests library is used to communicate with OpenRouter API. It sends structured recommendation prompts and receives AI-generated responses. Together, Scikit-learn and Requests support the strategic recommendation part of the project by

helping the system analyze title patterns and generate useful channel growth suggestions. Scikit-learn improves the quality of content analysis by identifying repeated and meaningful title terms. Requests makes API communication simple and reliable. These libraries help the dashboard provide smarter recommendations instead of only displaying basic statistics.

2.7 System Requirements

The hardware requirements for the YouTube Channel Performance and Engagement Analysis Dashboard are basic and suitable for normal development and testing. The system requires a computer or laptop with at least an Intel i3 or equivalent processor, 4 GB RAM, and minimum 2 GB free storage space. A stable internet connection is also required because the application fetches YouTube channel and video data using the YouTube Data API and may also connect to OpenRouter for AI-generated growth strategies.

The software requirements include an operating system such as Windows, Linux, or macOS. Python 3.9 or above is required as the main programming language. The project also requires Streamlit for building the dashboard interface, MySQL Server for storing channel and video data, and MySQL Workbench for database management. A code editor such as Visual Studio Code or PyCharm can be used for development. A modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox is required to run and view the Streamlit dashboard.

FEATURES & FUNCTIONALITY

The YouTube Channel Performance and Engagement Dashboard includes multiple features that help users collect data, analyze channel performance, understand audience engagement, compare channels, and generate useful growth suggestions. The features are organized into separate pages so that users can follow a simple and clear workflow without confusion. One of the main features of the application is flexible YouTube data input. Instead of requiring only a YouTube Channel ID, the system supports different input formats such as a YouTube channel link, video link, handle, or channel ID. This makes the application easier to use for common users who may not know how to find a channel ID manually. After receiving the input, the system identifies the correct channel and fetches the required public data using the YouTube Data API.[\[5\]](#)

The dashboard also includes a data collection and storage feature. It collects important channel information such as channel name, description, subscriber count, total views, video count, upload details, likes, comments, and video statistics. The collected data can be stored and reused for analysis, reducing the need to fetch the same information repeatedly. This improves the performance of the application and makes the analysis process more efficient. Another important feature is the channel profile display. After the data is fetched, the dashboard shows a clear profile of the selected YouTube channel, including the channel title, logo, description, subscriber count, total views, and number of uploaded videos. This gives users a quick overview of the channel before moving into detailed analysis.

The application provides performance metric calculation to help users understand how well a channel is performing. It calculates metrics such as average views per video, engagement rate, like ratio, comment ratio, subscriber-to-view ratio, and overall video performance. These metrics help users understand whether the channel is getting good audience interaction or needs improvement. The dashboard also contains interactive data visualizations. Graphs and charts are used to represent video views, likes, comments, upload trends, top-performing videos, engagement comparison, and performance over time. These visualizations make complex data easier to understand, especially for users who may not be comfortable reading only numbers or tables.

A channel comparison feature is also included in the system. Users can compare two or more YouTube channels based on subscribers, views, video count, engagement rate, and other important metrics. This helps users understand the strengths and weaknesses of different channels and identify areas where improvement is needed. The application also supports AI-assisted growth strategy generation. Using AI integration, the system can generate suggestions for improving channel performance. These suggestions may include ideas related to content planning, posting frequency, audience engagement, title improvement, thumbnail strategy, and video optimization. This feature helps creators understand what actions they can take to grow their channel.

Overall, the YouTube Channel Performance and Engagement Dashboard provides a complete solution for collecting, analyzing, visualizing, comparing, and improving YouTube channel performance using public YouTube data.[\[4\]](#)

3.1 Data Collection

The data collection feature is one of the most important modules of the YouTube Channel Performance and Engagement Dashboard. It allows users to collect public YouTube channel and video data in a simple and flexible way. Instead of asking users to enter only a channel ID, the system accepts different types of YouTube inputs, making the application more user-friendly for both technical and non-technical users.

Users can enter one or more YouTube inputs separated by commas. These inputs may include channel IDs, channel links, video links, YouTube handles, usernames, short links, or custom channel URLs. This flexibility helps users because many people do not know how to find a YouTube channel ID manually. For example, a user can paste a normal channel link or any video link from that channel, and the system can still identify the correct channel.

After the user submits the input, the system processes each value one by one. It removes unnecessary spaces, checks the input format, and identifies what type of YouTube input has been entered. The system then resolves each input into a valid YouTube channel ID. During this process, duplicate channel IDs are avoided so that the same channel is not processed multiple times. This improves efficiency and prevents repeated API calls.

Once the channel IDs are resolved, the system starts fetching channel information and video details using the YouTube Data API. The collected channel information includes the channel title, description, profile image, subscriber count, total

views, total video count, and uploads playlist ID. This information is useful for displaying the channel profile and creating a basic overview of the channel.

The application also collects video-related data from the channel. It identifies the uploads playlist of the selected channel and retrieves video IDs from that playlist. Since YouTube channels may contain many videos, the system uses pagination to collect video IDs page by page. This ensures that the application can collect data from channels with a large number of uploaded videos. After collecting video IDs, the application fetches detailed metadata and statistics for those videos. The metadata includes video title, description, publishing date, category, duration, thumbnail, and video URL. The statistics include view count, like count, and comment count. These details are later used for calculating performance metrics, generating graphs, comparing videos, and identifying top-performing content.

Another useful part of the data collection feature is comment collection. The system fetches comments from up to five videos with the highest comment counts, wherever comments are available. This approach helps reduce unnecessary API usage while still collecting useful engagement data. Comments are important because they help understand audience opinions, feedback, and interaction patterns. During the data collection process, the application shows progress feedback to the user. This helps users understand that the system is working and prevents confusion during longer data collection tasks. Progress messages may show steps such as resolving channel input, fetching channel details, collecting video IDs, fetching video statistics, and collecting comments.

3.1.1 Channel Resolution using YouTube API

Channel resolution is the process of converting different types of YouTube inputs into a valid YouTube channel ID. This is an important step because the YouTube Data API mainly works with channel IDs when fetching channel-level information. However, normal users may not always know the channel ID, so the application supports several user-friendly input formats. The channel resolution logic first checks whether the entered input is already a direct channel ID. YouTube channel IDs usually start with UC, so if the input matches this format, it can be used directly. If the input is not a direct channel ID, the system checks whether it is a YouTube handle, username, video URL, short link, or custom channel path. Comments are important because they help understand audience opinions, feedback, and interaction patterns.

If the user enters a YouTube handle, such as @channelname, the system uses the YouTube API to search and identify the matching channel. If the input is a username or custom channel link, the system extracts the required part from the URL and attempts to resolve it into a valid channel ID. This makes the system more flexible because YouTube channels can have different types of public URLs.

When a video link is provided, the system extracts the video ID from the URL. It supports normal YouTube video links as well as short links. After extracting the video ID, the application calls the YouTube API to fetch the video snippet. The snippet contains important details about the video, including the channel ID of the uploader. The system then uses this channel ID for further data collection.

This channel resolution process makes the application easier to use because users do not need to manually search for technical details. They can simply paste a link they already have, and the system automatically identifies the correct channel. It also reduces input errors and improves the overall user experience.[\[6\]](#)

3.1.2 Video Metadata Collection

Video metadata collection is the process of collecting detailed information about the videos uploaded by a YouTube channel. After the channel ID is resolved, the application fetches the uploads playlist ID from the channel details. Every YouTube channel has an uploads playlist that contains all uploaded videos. This playlist is used to collect the video IDs required for further analysis.

The application retrieves video IDs from the uploads playlist using pagination. Pagination is necessary because the YouTube API returns only a limited number of results in one request. If a channel has many videos, the system repeatedly requests the next page of results until all available video IDs are collected. This allows the dashboard to handle both small and large channels. After collecting video IDs, the application fetches video details in batches. Batch processing is used because the YouTube API allows multiple video IDs to be requested together. This reduces the number of API calls and improves the speed of data collection. It also helps manage API quota usage more efficiently.

The collected video metadata includes video title, description, publish date, category ID, duration, thumbnail URL, and video link. Along with metadata, the system also collects video statistics such as views, likes, and comments. These values are very Important because they are used to calculate engagement rate, average views, like

important because they are used to calculate engagement rate, average views, like ratio, comment ratio, and other performance metrics.

The publishing date helps the system analyze upload frequency and content trends over time. The duration helps compare short videos and long videos. The thumbnail helps display videos visually in the dashboard. The title and description are useful for content analysis, keyword understanding, and AI-based growth suggestions. The collected video metadata is later converted into structured data, usually in the form of a table or dataframe. This structured format makes it easier to filter, sort, analyze, and visualize the information. Users can view top-performing videos, compare engagement levels, identify upload patterns, and understand which type of content performs better.

3.2 Channel Dashboard and Metric Generation

The Channel Dashboard and Metric Generation module is used to present the collected YouTube data in a clear and meaningful way. After the data collection process is completed, users can select a channel and view its complete profile along with important performance indicators. This dashboard acts as the main analysis page where raw YouTube data is converted into useful insights.

The channel dashboard displays the selected channel profile with important details such as the channel thumbnail, channel name, description, subscriber count, total views, total videos, and channel creation date. This profile section gives users a quick overview of the channel before they move into detailed analysis. It helps users understand the basic identity and reach of the channel at a glance. . This dashboard acts as the main analysis page where raw YouTube data is converted into useful insights.

Along with the profile information, the dashboard also displays advanced metrics that summarize the overall performance of the selected channel. These metrics are calculated using the collected video statistics such as views, likes, comments, and subscriber count. Instead of showing only raw numbers, the dashboard converts the data into meaningful values that help users understand how well the channel is performing. The dashboard is useful for identifying the overall strength of a channel. For example, a channel may have many subscribers but low engagement, or it may have fewer subscribers but strong viewer interaction. By showing both profile details and calculated metrics together, the dashboard helps users understand the relationship between audience size, video reach, and engagement behavior.

3.2.1 KPI Calculation

KPI calculation is an important part of the dashboard because it converts raw YouTube statistics into performance indicators. KPI stands for Key Performance Indicator, which means a measurable value used to understand performance.

The dashboard calculates important KPIs such as average views per video, average engagement rate, and subscriber-view ratio. Average views per video shows how many views a channel receives on average for each uploaded video. This helps users understand the normal video performance of the channel. The average engagement rate is calculated using likes and comments compared with views. It helps users understand how actively viewers are responding to the content. A higher engagement rate means that viewers are not only watching the videos but also interacting with them through likes and comments.

The subscriber-view ratio helps compare the relationship between the number of subscribers and total views. This metric shows whether the channel's content is reaching viewers effectively compared to its subscriber base. If a channel has many subscribers but low views, it may indicate low audience activity. If a channel has fewer subscribers but high views, it may indicate strong content reach or viral performance. These KPIs help users compare different channels, analyze video performance, and identify areas for improvement. They also make the dashboard more useful for creators, marketers, and analysts who want to understand YouTube performance in a simple way.

3.2.2 Category and Upload Feature Engineering

Category and upload feature engineering is used to create additional useful fields from the collected video data. These derived fields help the system perform deeper analysis of content type, upload timing, and publishing behavior. The application enriches each video record with fields such as title length, upload hour, upload day, and category label. Title length is calculated from the number of characters or words in the video title. This can help users understand whether shorter or longer titles perform better.

Upload hour is extracted from the publishing time of the video. This helps analyze at what time videos are usually uploaded and whether upload timing affects views or engagement. Upload day is used to identify which days of the week the channel posts videos most often. When category IDs are available, the system maps them to readable category labels such as Entertainment, Education, Music, Gaming, Sports, or News. This makes the data easier to understand because users can see the News. Makes

the data easier to understand actual category name instead of only a numeric category ID. These engineered features are useful for creating graphs and deeper insights. For example, the dashboard can show which upload days receive better engagement, which category performs best, or whether title length affects video views. This improves the analytical value of the system.[\[7\]](#)

3.2.3 Paginated Video Table

The dashboard includes a paginated video table that displays detailed information about the videos of the selected channel. This table allows users to explore video-level data in an organized format. The table includes important fields such as video ID, watch link, thumbnail preview, title, description, publish date, views, likes, and comments. The watch link allows users to directly open the video on YouTube. The thumbnail preview helps users visually identify the video. The title and description provide content information, while views, likes, and comments show performance details.

Pagination is used to improve usability when a channel contains many videos. Without pagination, displaying hundreds or thousands of videos on one page would make the dashboard slow and difficult to use. Pagination divides the video records into smaller pages, making the table easier to read and navigate. Users can move between pages to explore more videos without overwhelming the screen. This makes the dashboard more user-friendly and improves performance. The paginated table also helps users compare videos, identify top-performing content, and review detailed video statistics.

3.3 Visualization and Engagement Analysis

The Visualization and Engagement Analysis page is designed to convert collected YouTube data into easy-to-understand graphical insights. Instead of showing only numbers in tables, this module uses charts and visual elements to help users understand channel performance more clearly. Visual analysis is useful because it allows users to quickly identify trends, patterns, growth periods, and engagement behavior.

On this page, users can select a channel from the available collected data. After selecting the channel, the system displays different types of visual insights related to views, uploads, and engagement. The page is divided into tabbed sections such as basic insights, advanced insights, and custom metrics. This tab-based structure keeps the interface clean and organized. The basic insights section focuses on simple performance charts such as views over time and top-performing videos. The advanced insights

section provides deeper analysis such as engagement trends and relationships between views, likes, and comments. The custom metrics section allows users to select specific metrics and time periods for analysis.

This visualization page helps users understand how a channel is performing over time. It also helps identify which videos performed well, which time periods showed growth, and how viewers interacted with the content. For creators and analysts, these insights are useful for planning future content and improving channel strategy. The paginated table also helps users compare videos, identify top-performing content, and review detailed video statistics.

3.3.1 Views and Upload Analysis

The Views and Upload Analysis section focuses on understanding how videos perform in terms of views and upload activity. The system displays important charts such as views over time, top ten most viewed videos, and video distribution by year. The views over time chart shows how video views are distributed according to publishing dates. This helps users identify growth periods and performance changes. For example, if many videos published during a particular month or year received high views, it may indicate a successful content strategy or trending topic during that period.

The top ten most viewed videos chart highlights the best-performing videos of the selected channel. This is useful because it helps users understand which content attracted the highest audience attention. By analyzing these videos, users can identify common patterns such as popular topics, effective titles, attractive thumbnails, or suitable video duration. The video distribution by year chart shows how many videos were uploaded in each year. This helps users understand the channel's upload consistency and activity level.

A channel with regular uploads may show steady content production, while gaps in upload years may indicate inactive periods. Together, these charts help users answer important questions such as which videos performed best, when the channel experienced growth, and how consistent the channel has been in uploading content.

3.3.2 Engagement Analysis

The Engagement Analysis section focuses on how viewers interact with the channel's videos. Engagement is calculated using likes and comments relative to views. This means the system does not only check how many people watched a video, but also how many viewers responded to it. The dashboard calculates engagement by considering

likes and comments as viewer interactions. A video with high views but low likes and comments may indicate passive viewing, while a video with strong likes and comments shows active audience response. This helps users understand the quality of audience interaction.

The visualization page includes charts such as views versus engagement scatter plots, monthly engagement trends, and an engagement funnel from views to likes to comments. The views versus engagement scatter plot helps compare video reach with audience interaction. It can show whether highly viewed videos also have strong engagement or whether some videos have fewer views but better interaction. The monthly engagement trend chart shows how engagement changes over time. This helps users identify months where audience interaction was high or low.

The engagement funnel shows the flow from views to likes to comments. It helps users understand how many viewers move from simply watching a video to actively liking or commenting on it. This funnel is useful because comments usually require more effort than likes, so they represent deeper audience involvement. Overall, engagement analysis helps users understand viewer behavior more effectively and supports better content planning.

3.3.3 Plotly Integration

The dashboard uses Plotly for creating interactive charts inside the Streamlit application. Plotly is useful because it provides modern, responsive, and interactive visualizations that improve the user experience. Plotly charts are embedded directly into Streamlit pages, allowing users to explore data visually without leaving the application. These charts support hover details, dynamic rendering, zooming, and responsive layouts. When users move the cursor over a chart element, they can see additional information such as exact views, likes, comments, dates, or video titles.

The interactive nature of Plotly makes analysis easier compared to static charts. Users can explore the data in more detail and understand specific values without reading large tables. This is especially helpful when analyzing many videos or long-term channel performance. Plotly also improves the appearance of the dashboard. The charts are clean, colorful, and easy to interpret. They help make the application more professional and suitable for academic, analytical, and presentation purposes. Engagement analysis helps users understand viewer behavior more effectively and supports better content.

In conclusion, Plotly integration makes the visualization page more powerful and user-friendly. It allows users to interact with YouTube data, understand trends clearly, and make better decisions based on visual insights.

3.4 Global Filter Feature

The Global Filter Feature is an important part of the YouTube Channel Performance and Engagement Dashboard because it allows users to analyze selected portions of data instead of viewing the complete dataset every time. When a YouTube channel contains many videos, it becomes difficult to study all records together. Global filters solve this problem by allowing users to narrow the data based on specific conditions.

Global filters can be applied to both dashboard and visualization pages. This means that when a user applies a filter, the displayed metrics, tables, and charts are updated according to the selected filtered data. This makes the analysis more focused, accurate, and useful. The system allows users to filter videos using different options such as search keyword, view range, publish date range, engagement level, and duration group. These filters help users inspect specific video segments based on their requirements. For example, a user can search for videos related to a particular topic, view only high-performing videos, analyze videos uploaded during a specific time period, or compare short and long videos.

The search keyword filter allows users to search videos by title or description. This is useful when users want to analyze videos related to a specific content theme. For example, if a channel uploads different types of videos, users can search a keyword and study only those videos. The view range filter helps users classify videos based on performance. View categories include Low, Medium, High, and Viral. These categories make it easier to understand video reach without manually checking each view count. Low-view videos may show content that needs improvement, while viral videos help identify successful content patterns.

The publish date range filter allows users to select videos uploaded within a specific date range. This is useful for analyzing monthly, yearly, or campaign-based performance. Users can compare how videos performed during different time periods and identify growth or decline patterns. The engagement level filter classifies videos into Low, Medium, and High engagement levels based on percentage thresholds. Engagement is calculated using likes and comments compared with views. This helps users identify videos where viewers interacted more actively with the content.

The duration group filter classifies videos as Short, Medium, or Long based on video duration in seconds. This helps users compare how video length affects performance. For example, users can analyze whether short videos receive more views or whether longer videos receive better engagement. Global filters can be applied to both dashboard and visualization pages. This means that when a user applies a filter, the displayed metrics, tables, and charts are updated according to the selected filtered data. This makes the analysis more focused, accurate, and useful.

3.5 AI-Powered Assistance Features

The AI-Powered Assistance Features add an intelligent recommendation layer to the YouTube Channel Performance and Engagement Dashboard. This section helps users move beyond basic data analysis and supports them in making better decisions for future content planning. Instead of only showing charts and numbers, the system studies stored video data and identifies useful content patterns. The growth strategy section reviews video performance, title patterns, engagement behavior, and topic trends. Based on this analysis, it provides suggestions that can help creators improve their channel growth. These suggestions may include what type of topics to continue, which content areas need improvement, how to improve publishing consistency, and how to increase viewer engagement.

3.5.1 Growth Strategy Insights

The Growth Strategy Insights feature identifies high-performing and low-performing videos using percentile-based analysis. The system uses the 80th percentile of view count to identify strong-performing videos and the 20th percentile of view count to identify weak-performing videos. Videos above the 80th percentile are considered high-performing because they received more views than most other videos on the channel. Videos below the 20th percentile are considered low-performing because they received fewer views compared to the majority of videos.

After identifying these groups, the system analyzes video titles to extract working topics and weaker topics. Working topics are themes commonly found in high-performing videos, while weaker topics are themes found in low-performing videos. This helps users understand which content ideas are attracting more viewers and which ideas may need improvement. The system then prepares strategy suggestions based on these findings. For example, it may suggest creating more videos around successful. This section helps users move beyond basic data analysis and supports them in making.

better decisions for future content planning topics, improving titles for weaker topics, experimenting with better thumbnails, or adjusting upload timing.

3.5.2 OpenRouter AI Recommendation

The OpenRouter AI Recommendation feature is used to generate AI-based growth suggestions. When an OpenRouter API key is configured in the project, the system sends a structured evidence pack to the selected AI model. This evidence pack may include channel summary, top-performing videos, low-performing videos, average views, engagement rate, upload patterns, and extracted topic phrases. The AI model uses this information to generate channel-specific recommendations.

The AI response can include content suggestions, topic ideas, publishing advice, audience engagement strategies, and improvement actions. These recommendations are not random; they are generated using the channel's actual stored data.

For example, the AI may suggest focusing on topics that already performed well, improving video titles, posting more consistently, creating comparison-based content, or encouraging viewers to comment. This makes the growth strategy section more personalized and useful. The search keyword filter allows users to search videos by title or description. This is useful when users want to analyze videos related to a specific content theme

3.5.3 Local Recommendation Fallback

The Local Recommendation Fallback feature ensures that the dashboard remains useful even when AI access is unavailable. Sometimes the OpenRouter API may fail because of internet issues, invalid API key, quota limits, or external service problems.

In such cases, the system does not stop working. Instead, it uses local rule-based logic to generate recommendations. These local recommendations are created using stored video data, performance metrics, engagement values, and topic extraction results. For example, if engagement is low, the system may suggest improving call-to-action lines, asking questions in videos, or encouraging comments. If upload frequency is low, it may suggest maintaining a regular posting schedule. If certain topics perform well, it may suggest creating more content around those topics.[\[8\]](#)

3.5.4 Topic Extraction

The Topic Extraction feature helps identify common phrases and themes from video titles. The project uses TF-IDF based phrase extraction to detect important words or phrases that appear in video titles. TF-IDF stands for Term Frequency–Inverse

Document Frequency. It helps identify words that are important in a group of titles by checking how often they appear and how unique they are compared to other titles.

In this project, TF-IDF is used to extract topic phrases from high-performing and low-performing videos. These phrases help summarize what type of content is working well and what type of content is performing weakly. For example, if high-performing videos commonly contain phrases related to tutorials, reviews, or trending topics, the system can identify them as useful content directions. If low-performing videos contain repeated weak topics, the system can suggest improving or changing those content areas. Topic extraction makes the AI recommendations more evidence-based and improves the quality of strategy generation.

3.5.5 Performance Comparison

The Performance Comparison feature allows users to compare selected YouTube channels side by side. This is useful when users want to understand how one channel performs compared to another channel in the same niche or category. The comparison section provides a leaderboard, benchmark table, subscriber chart, total view chart, upload volume chart, and performance share chart.

The leaderboard ranks channels based on selected performance values. The benchmark table displays important metrics such as subscribers, total views, video count, average views, and engagement rate. The subscriber chart compares audience size, while the total view chart compares overall reach. The upload volume chart shows how active each channel is in terms of video uploads. The performance share chart helps users understand how much contribution each channel has in the overall comparison group. This feature is useful for competitive analysis. It helps users identify which channel has stronger reach, better engagement, higher upload consistency, or better overall performance.

3.5.6 Help and FAQ

The Help and FAQ page is included to guide users in understanding and using the dashboard properly. Since the project contains multiple sections, metrics, filters, charts, and AI features, the Help page makes the application easier to use. This page explains how to use each dashboard section, including data collection, channel dashboard, visualization page, global filters, growth strategy, and comparison features. It also explains important metrics such as views, likes, comments, engagement rate, average views, upload volume, and performance share.

The FAQ section helps users understand common questions, such as how to enter a YouTube link, why some comments may not be available, how engagement rate is calculated, and what AI recommendations mean. By providing clear explanations, the Help and FAQ page improves user experience and reduces confusion. It is especially useful for beginners who may not be familiar with YouTube analytics or technical terms. Overall, the AI-powered assistance features make the project more intelligent, reliable, and user-friendly.

3.6 Technical Capabilities

The Technical Capabilities section describes the important backend and system-level features that make the YouTube Channel Performance and Engagement Dashboard flexible, reliable, and easy to use. These capabilities support data collection, API communication, data conversion, navigation, configuration management, and error handling. Together, they improve the overall performance and usability of the application.

The project is built using Streamlit, Python libraries, YouTube Data API, data processing techniques, and optional AI integration through OpenRouter. These technical features allow the application to collect public YouTube data, process into structured form.

3.6.1 Multiple Input Support

The system supports multiple YouTube input formats, including channel IDs, channel URLs, handles, usernames, video URLs, and comma-separated inputs. This makes the data collection process flexible for different types of users. Many users may not know how to find a YouTube channel ID. Because of this, the application allows them to paste a normal YouTube channel link, a video link, or a handle such as @channelname. The system then resolves the input and identifies the correct channel ID automatically.

The comma-separated input feature allows users to enter multiple channels at once. This is useful when users want to collect data for more than one channel or compare channels later. The system processes each input separately and avoids duplicate channel IDs. This feature improves user convenience and reduces manual effort.

3.6.2 Batch API Processing

The project uses batch API processing to improve efficiency while working with the YouTube Data API. After resolving a channel, the system collects video IDs from the

channel's uploads playlist. Since a channel may contain many videos, the system uses pagination to collect video IDs page by page. After collecting video IDs, the application requests video details in batches instead of sending one API request for each video.

Batch processing helps reduce the number of API calls and follows YouTube API limits. It also improves the speed of data collection and makes the system more suitable for channels with many uploaded videos. This approach makes the backend more optimized and reliable.

3.6.3 Duration and Date Conversion

The project performs important data conversion tasks to make YouTube data suitable for analysis. YouTube video durations are received in ISO 8601 format, such as minutes and seconds format used by the API. The system converts these durations into total seconds. This conversion helps classify videos into duration groups such as Short, Medium, and Long. It also supports duration-based filtering and comparison of video performance based on length.

The project also converts published timestamps into Python datetime values. YouTube provides publishing dates in timestamp format, and converting them into datetime values allows the system to perform time-based analysis. These converted dates are used for charts such as views over time, upload distribution by year, monthly engagement trends, and publish date filters. This conversion helps classify videos into duration groups such as Short, Medium, and Long.

3.6.4 Workspace Navigation

The application uses the Streamlit sidebar to provide consistent workspace navigation. The sidebar allows users to move between different pages of the dashboard easily. Main pages include Data Collection, Channel Dashboard, Visualizations, Comparison, Growth Strategy, and Help/FAQ. This structure keeps the application organized and makes the workflow easy to follow.

Users can first collect data, then view the dashboard, apply filters, explore visualizations, compare channels, and finally generate growth strategies. The sidebar improves usability because users do not need to search for pages manually. This navigation system makes the application simple and professional.

3.6.5 Environment Integration

The project uses environment variables to manage API keys and model settings. Sensitive information such as the YouTube Data API key and OpenRouter API key is

stored in a .env file instead of being written directly in the source code. This improves security because API keys are kept separate from the main program files. It also makes the project easier to configure. If the user wants to change an API key or AI model, they can update the environment file without modifying the source code.

Environment integration also supports deployment because different systems can use different configuration values. For example, the developer can use one API key during testing and another during deployment. This makes the application more flexible and secure.

3.6.6 Error Handling

The project includes error handling to make the application more stable and user-friendly. Since the system depends on external APIs and user-provided inputs, errors can occur during data collection and analysis. The application provides error messages for unresolved inputs, missing channels, unavailable videos, disabled comments, empty charts, invalid API responses, and OpenRouter failures. These messages help users understand what went wrong and what action they can take next. For example, if a user enters an invalid YouTube link, the system shows a message explaining that the input could not be resolved. If comments are disabled on a video, the system skips comment collection and continues processing other available data.

If the OpenRouter API fails, the system can still generate local rule-based recommendations. This prevents the growth strategy page from becoming unusable. Overall, error handling improves reliability, prevents sudden application failure, and gives users a smoother experience while using the dashboard.

The system supports multiple YouTube input formats, including channel IDs, channel URLs, handles, usernames, video URLs, and comma-separated inputs. This makes the data collection process flexible for different types of users. Many users may not know how to find a YouTube channel ID. Because of this, the application allows them to paste a normal YouTube channel link, a video link, or a handle such as @channelname. The system then resolves the input and identifies the correct channel ID automatically.

The comma-separated input feature allows users to enter multiple channels at once. This is useful when users want to collect data for more than one channel or compare channels later. The system processes each input separately and avoids duplicate channel IDs. This feature improves user convenience and reduces manual effort.

SYSTEM WORKFLOW & PROCESSING

The YouTube Channel Performance and Engagement Dashboard follows a structured workflow from user input to final insight display. Each stage of the system performs a specific task and passes clean and organized data to the next stage. This step-by-step workflow makes the application easier to understand, test, debug, and maintain. The workflow begins with the data collection stage, where the user enters one or more YouTube inputs. These inputs may include channel IDs, channel links, video links, handles, usernames, or custom channel URLs. The system accepts comma-separated inputs, which allows users to collect data for multiple channels at the same time.

After receiving the input, the system performs input resolution. In this stage, the application identifies the type of input provided by the user. If the input is a direct channel ID, it is used directly. If the input is a video link, the system extracts the video ID and finds the related channel ID. If the input is a handle, username, or custom URL, the system resolves it into a valid channel ID using the YouTube API. Once valid channel IDs are obtained, the system moves to the API fetching stage. The application fetches channel details such as channel name, description, thumbnail, subscriber count, total views, total videos, and creation date. It also fetches the uploads playlist ID of the channel. From this playlist, the system collects video IDs and then retrieves detailed video metadata and statistics.

The next stage is data conversion and preprocessing. The raw data received from the YouTube API is converted into a usable format. Video durations are converted from ISO 8601 format into seconds, and publishing timestamps are converted into Python datetime values. Category IDs are mapped to readable category names where available. Additional fields such as upload day, upload hour, title length, duration group, view category, and engagement level are also generated. After preprocessing, the cleaned data is stored through the database insertion stage. Channel details, video metadata, statistics, and comment data are saved so that the application can reuse the information without fetching it repeatedly. This improves efficiency and supports later analysis, comparison, and recommendation generation.

Once data is available, the system applies global filters based on user selection. Users can filter videos by keyword, view range, publish date range, engagement level,

and duration group. These filters help narrow the dataset and allow users to focus on specific video segments. The filtered data is then used for metric calculation. The system calculates important KPIs such as average views per video, engagement rate, subscriber-view ratio, upload volume, and performance share. These metrics convert raw numbers into meaningful performance indicators. After metric calculation, the system displays insights through visualization and dashboard pages. Charts are generated for views over time, top videos, upload distribution, engagement trends, and comparison analysis. Interactive Plotly charts inside Streamlit allow users to explore the data visually.

The workflow also includes channel comparison, where multiple selected channels can be reviewed side by side using leaderboards, benchmark tables, and charts. Finally, the system performs recommendation generation using AI-powered suggestions through OpenRouter or local fallback rules.[\[9\]](#)

4.1 Input File Handling Process

In this project, the input handling process refers to YouTube input handling rather than local file upload. The system does not require users to upload CSV, Excel, or text files for collecting channel data. Instead, users provide YouTube-related inputs directly on the Data Collection page. These inputs may include channel IDs, channel URLs, YouTube handles, usernames, video links, short links, custom channel links, or comma-separated values. The workflow also includes channel comparison, where multiple selected channels can be reviewed side by side using leaderboards, benchmark tables.

This approach makes the project easier to use because users do not need to prepare any external files before using the dashboard. They can simply copy and paste a YouTube link or channel identifier into the input box. This is especially helpful for common users who may not know technical details such as channel IDs. The input handling process starts when the user enters one or more values into the input field. If the user wants to collect data for multiple channels, they can separate the inputs using commas. The system reads the complete input string and splits it into separate values. Each value is then processed individually. After splitting the input, the system removes unnecessary spaces from the beginning and end of each value. This process is called trimming. Trimming is important because users may accidentally enter extra spaces before or after a link or channel ID. If these spaces are not removed, the system may fail to recognize the input correctly.

The system also removes empty values. Empty values may occur when the user adds extra commas or leaves blank spaces between inputs. For example, if the user enters two commas by mistake, the system ignores the empty part and continues processing the valid values. This prevents unnecessary errors and improves the reliability of the input handling process. Once the inputs are cleaned, the system prepares them for channel resolution. Channel resolution means converting different input formats into valid YouTube channel IDs. The YouTube channel ID is required because most YouTube API operations use it to fetch channel-level data.

During this process, the system checks whether each input is valid or invalid. Valid inputs continue through the processing pipeline, while invalid inputs are reported to the user. If an input cannot be recognized or resolved, the system displays an error or warning message explaining that the input could not be processed. This input validation step is important because it prevents the application from crashing due to incorrect input. It also helps users correct their mistakes and try again. For example, if a user enters an incomplete YouTube link, the system can notify them that the input is invalid.

The input handling process also avoids duplicate processing. If the same channel is entered more than once, the system identifies the duplicate channel ID and prevents repeated API calls. This saves time, reduces API usage, and improves system performance. Overall, the input handling process is designed to be flexible, user-friendly, and error-resistant. It allows users to provide YouTube data in different formats, cleans the input values, removes invalid or empty entries, avoids duplicates, and prepares valid inputs for the next stage of processing. This input validation step is important because it prevents the application.

4.2 Source Data Parsing Sequence

The source data parsing sequence is the process of identifying the type of YouTube input entered by the user and extracting the required information from it. Since users can enter different formats such as channel IDs, handles, usernames, custom links, or video links, the system must correctly understand each input type before fetching data. The parsing sequence begins by checking whether the input is already a valid channel ID. YouTube channel IDs usually follow a specific format and commonly start with UC. If the input matches the expected channel ID pattern, the system treats it as a direct channel ID and sends it to the next processing stage.

If the input is not a direct channel ID, the system checks whether it is a YouTube handle. A handle usually starts with the @ symbol, such as @channelname. Handles are easier for users to recognize, so supporting them makes the system more user-friendly. When a handle is detected, the system uses YouTube API lookup or search methods to find the related channel ID. The next check is for username-based URLs. Some older YouTube channels use username links instead of handles or channel IDs. These URLs may contain paths such as /user/username. The system extracts the username from the URL and uses YouTube API channel lookup to resolve it into a channel ID.

The system also supports custom channel links. Custom links may contain paths such as /c/channelname or other channel-specific custom URLs. Since custom URLs may not directly provide a channel ID, the system performs a channel search to find the most relevant matching channel. The best result is then selected and used for data collection. For video links, the parsing process works differently. If the user enters a normal YouTube video URL or a short YouTube link, the system extracts the video ID from the link. YouTube video IDs are unique identifiers assigned to each video. After extracting the video ID, the system calls the YouTube API to fetch the video snippet. The video snippet contains the channel ID of the video owner. This channel ID is then used for channel data collection.

URL parsing and regular expressions are used throughout this process. URL parsing helps separate different parts of a link, such as domain, path, and query parameters. Regular expressions help detect patterns such as channel IDs, video IDs, handles, and URL formats. This parsing sequence allows the system to support many different user input styles. For example, one user may enter a direct channel ID, another may paste a video link, and another may enter a handle. The system can process all these formats and convert them into a common channel ID format.

If the system cannot identify or resolve an input, it marks that input as invalid and informs the user. This prevents incorrect data collection and helps users understand what went wrong. Valid inputs continue to the next stage, where channel details and video data are fetched from the YouTube API. The parsing sequence is important because it acts as a bridge between user-friendly input and API-required identifiers. It makes the application easier to use while still maintaining technical accuracy in the backend. The source data parsing sequence improves the flexibility and reliability.

4.3 Internal Data Processing

After the required data is fetched from YouTube, the system converts the raw response data into Pandas DataFrames. This step is important because API responses are usually received in nested JSON format, which is not directly suitable for analysis, filtering, or visualization. Pandas DataFrames provide a structured tabular format where each row represents a channel or video record and each column represents a specific field.

The system separates channel-level data and video-level data into different structured datasets. Channel fields such as channel ID, channel name, description, subscriber count, total views, video count, thumbnail, and creation date are normalized into consistent column names. Similarly, video fields such as video ID, title, description, publish date, duration, views, likes, comments, category ID, and thumbnail are also arranged into standard columns. Normalization is useful because YouTube API responses may contain nested keys and different naming styles. By converting them into consistent column names, the dashboard can access the data easily across different pages. This also makes the code cleaner and reduces errors during metric calculation and chart generation.

The system performs several important data conversion operations. Published timestamps are converted into Python datetime values so that they can be used for date filters, time-based charts, upload trends, and monthly or yearly analysis. Video durations received from YouTube are in ISO 8601 format, so they are converted into total seconds. This conversion helps classify videos into duration groups such as Short, Medium, and Long. Numeric statistics such as views, likes, comments, subscriber count, total views, and video count are converted into integer values. This is necessary because API values may be received as strings. If these values are not converted, mathematical operations such as averages, ratios, percentages, and comparisons may not work correctly.

During dashboard display, the system generates additional derived fields. These include engagement rate, title length, upload hour, upload day, category labels, and video URLs. Engagement rate is calculated using likes and comments relative to views. Title length is calculated from the number of characters or words in the video title. Upload hour and upload day are extracted from the publish date to support upload pattern analysis. Category labels are created by mapping category IDs to readable category names whenever category information is available. This makes the dashboard

easier to understand because users can view category names instead of numeric IDs. Video URLs are also generated so users can directly open videos from the dashboard table.

These processed and derived fields are used across different parts of the project. Filters use fields such as publish date, duration group, engagement level, and view category. Charts use fields such as views, likes, comments, engagement rate, upload day, and publish date. Tables use video URLs, thumbnails, titles, descriptions, and statistics. Overall, internal data processing converts raw API responses into clean, structured, and analysis-ready data. This step is essential because accurate dashboards, visualizations, comparisons, and recommendations depend on properly processed data. These processed and derived fields are used across different parts of the project.

4.4 API Request and Response Cycle

The API Request and Response Cycle describes how the system communicates with external services to collect YouTube data and generate AI-based recommendations. The project mainly uses the YouTube Data API for collecting channel and video information, and OpenRouter API for AI-powered growth strategy generation. The YouTube API request cycle starts with authenticated client creation. The system reads the YouTube API key from the environment file and uses it to create an authenticated API client. Storing the key in an environment file keeps sensitive information separate from the main source code and improves security.

After authentication, the system sends requests to different YouTube API endpoints depending on the required operation. The channels endpoint is used to fetch channel details such as title, description, thumbnail, subscriber count, total views, video count, creation date, and uploads playlist ID. The search endpoint is used when the system needs to resolve handles, usernames, or custom channel links. It helps find the most relevant channel when a direct channel ID is not available.

The playlistItems endpoint is used to collect video IDs from the uploads playlist of a channel. Since YouTube may return limited results per request, pagination is used to collect all available video IDs page by page. The videos endpoint is used to fetch detailed video metadata and statistics. This includes video title, description, publish date, category ID, duration, views, likes, comments, and thumbnails. Video details are requested in batches to reduce API calls and improve efficiency. The system reads the YouTube API key from the environment file and uses it to create.

The commentThreads endpoint is used to fetch comments from selected videos. In this project, comments are collected from up to five videos with the highest comment counts, where comments are available. This reduces API usage while still providing useful audience feedback data. After each API request, the system receives a response in JSON format. The response is checked for required fields, converted into structured data, and passed to the internal processing stage. If a response is empty, invalid, or missing important fields, the system displays a suitable error message and continues wherever possible.

The OpenRouter request cycle is used only for strategy generation. When the user opens the growth strategy section and an OpenRouter API key is configured, the system prepares a structured payload for the selected AI model. This payload contains channel evidence such as top-performing videos, low-performing videos, engagement metrics, extracted topics, upload patterns, and summary statistics. It also includes the selected model name, token limits, and prompt text. The prompt guides the AI model to generate useful channel-specific growth recommendations.

If the configured model fails due to quota issues, invalid model access, or API errors, the system may attempt fallback models according to the environment settings. This improves reliability and prevents the strategy feature from failing completely. If OpenRouter access is unavailable, the system uses local rule-based recommendations. This ensures that users still receive useful suggestions even without AI access.

In conclusion, the API request and response cycle allows the project to collect live public YouTube data and generate intelligent recommendations. Proper authentication, endpoint selection, batch processing, response validation, and fallback handling make the system efficient and dependable.

4.5 Analysis Result Processing

Analysis result processing is performed according to the selected page of the application. After data is collected, cleaned, and stored, the system uses it to generate different types of outputs such as KPIs, tables, charts, comparisons, and recommendations. Each page processes the available data in a different way based on its purpose. On the Dashboard page, the system focuses on channel-level and video-level performance. It retrieves the selected channel data from the database and calculates important KPIs such as average views per video, engagement rate, subscriber-view ratio, total views, total videos, and upload performance. These values

are displayed in a simple format so users can quickly understand the overall condition of the channel.

The dashboard also displays video records in a structured table. The video table includes details such as video ID, title, description, publish date, views, likes, comments, thumbnail, and watch link. Pagination is applied so that large numbers of videos can be viewed easily without slowing down the interface. On the Visualization page, the system processes video records for chart generation. Records are grouped by month, year, upload day, or selected metric depending on the selected chart. For example, views can be grouped by publishing date to show views over time, while video count can be grouped by year to show upload distribution. Engagement values can also be grouped by month to show engagement trends.

The analysis result processing workflow can be summarized as follows:

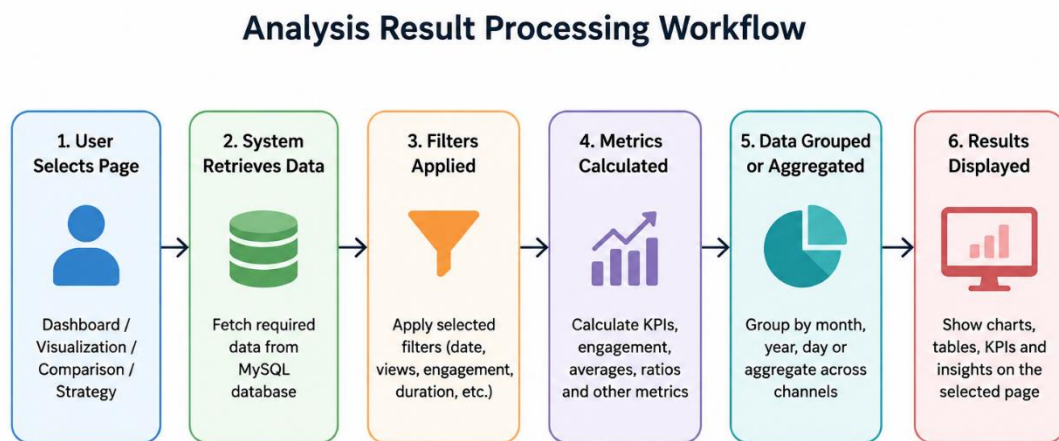


Figure 4.5: Analysis Result Processing Workflow

The visualization page uses processed data to create charts such as views over time, top ten most viewed videos, video distribution by year, views versus engagement scatter plot, monthly engagement trend, and engagement funnel. These charts help users understand trends and patterns more easily than raw tables. On the Comparison page, the system aggregates channel-level values across selected channels. It compares subscribers, total views, video count, average views, engagement rate, and upload volume. The comparison output may include a leaderboard, benchmark table, subscriber chart, total view chart, upload volume chart, and performance share chart. On the Visualization page, the system processes video records for chart generation. Records are grouped by month, year, upload day, or selected metric depending.

This page is useful for comparing multiple channels side by side. It helps users understand which channel has better reach, stronger engagement, higher upload activity, or better overall performance.

User selects page → System retrieves required data → Filters are applied → Metrics are calculated → Data is grouped or aggregated → Charts/tables are generated → Results are displayed on dashboard. This workflow ensures that the application displays accurate and page-specific results. It also makes the system easier to maintain because each page has a clear processing responsibility.

4.6 Report Compilation Process

The application currently presents reports through live dashboard screens instead of generating a separate export file from inside the app. This means that the dashboard itself works as an interactive report for the selected YouTube channel. The report content includes channel profile information, key performance indicators, video tables, charts, comparison results, and strategy suggestions. Together, these elements provide a complete view of channel performance and engagement behavior.

Unlike a static report, the live dashboard allows users to interact with the data. Users can select channels, apply filters, change date ranges, view charts, compare channels, and explore strategy recommendations. This makes the report more flexible and useful than a fixed document. The dashboard report includes multiple sections. The channel dashboard shows basic channel details and KPIs. The visualization page shows graphical insights. The comparison page shows benchmark results across channels. The growth strategy page provides AI-assisted or rule-based recommendations. This interactive report format is useful during project demonstrations, academic presentations, and analysis sessions because users can directly explore the data inside the application.

In the future, the report compilation process can be extended by adding export options such as PDF, Word, or Excel reports. A PDF report could be useful for academic submission or formal documentation. A Word report could allow users to edit and customize the content. An Excel report could help users perform further calculations and offline analysis. Such export functionality would allow users to save dashboard . The visualization page shows graphical insights for academic submission, business review, creator planning, client reporting. It would also make the project more professional and practical for real-world use.

4.7 Data Storage and Retrieval

The project uses MySQL tables for persistent data storage. Persistent storage means that collected channel and video data remains available even after the application is closed. This avoids the need to fetch the same data repeatedly from the YouTube API. The database stores channel details, video metadata, engagement statistics, and selected comment data. This stored data is later retrieved for dashboard display, visualization, comparison, filtering, and recommendation generation.

The database is accessed using SQLAlchemy functions. SQLAlchemy helps connect the Python application with the MySQL database and allows the system to perform insert, update, and retrieve operations efficiently. The project mainly uses three tables: channels, videos, and comments. The channels table stores channel profile and current channel-level performance. It includes fields such as channel ID, channel name, description, created date, subscriber count, total views, total videos, and thumbnail URL. This table is used to display the channel profile and calculate channel-level comparison metrics.

The videos table stores video-level metadata and engagement statistics. It includes fields such as video ID, channel ID, title, description, published date, duration, duration seconds, view count, like count, comment count, and thumbnail URL. This table is used for video analysis, charts, filters, KPI calculation, and strategy generation.

The comments table stores selected top-level comments when comments are available. It includes fields such as comment ID, video ID, comment text, author name, and published date. These comments can be used to understand audience feedback and engagement behavior.

Insert operations use ON DUPLICATE KEY UPDATE logic. This means that if the same channel or video already exists in the database, the system updates the existing record instead of creating a duplicate entry. This is important because YouTube statistics such as views, likes, comments, subscribers, and thumbnails can change over time. For example, if a user collects data for the same channel again after a few days, the system refreshes the updated view count, like count, comment count, and subscriber count. This keeps the database current while avoiding duplicate records. . This avoids the need to fetch the same data repeatedly from the YouTube API. The database stores channel details, video metadata, engagement statistics, and selected comment data. This stored data is later retrieved for dashboard display.

The table structure can be represented as follows:

Table	Main Fields	Use
channels	channel_id, channel_name, description, created_date, subscriber_count, total_views, total_videos, thumbnail_url	Stores channel profile and current channel-level performance.
videos	video_id, channel_id, title, description, published_date, duration, duration_seconds, view_count, like_count, comment_count, thumbnail_url	Stores video-level metadata and engagement statistics.
comments	comment_id, video_id, comment_text, author_name, published_at	Used by the application logic for storing selected top-level comments when available.

Table 4.7: MySQL Tables

Overall, the data storage and retrieval process makes the application efficient, reliable, and reusable. It supports repeated analysis, avoids unnecessary API calls, keeps data updated, and provides a strong backend foundation for the dashboard. [\[10\]](#)

4.8 Execution Sequence of the System

The execution sequence of the system starts when the Streamlit application is loaded by the user. At the beginning, the application applies the page configuration such as page title, layout, and sidebar settings. After that, the session state is initialized to store temporary values during the running session.

The sidebar is then rendered to provide navigation between different pages of the application. Users can select pages such as Data Collection, Channel Dashboard, Visualization, Comparison, Growth Strategy, and Help/FAQ. Based on the selected option, the related page function is executed. Each page loads only the data required for its own interface. This makes the application more organized and avoids unnecessary processing. For example, the dashboard page loads channel and video data, while the comparison page loads data for multiple selected channels.

For the Data Collection page, the execution sequence starts with input validation. The system checks whether the user has entered valid YouTube inputs. After validation, the system performs channel resolution to convert channel links, handles,

usernames, or video links into valid channel IDs. After resolving the channel ID, the system fetches channel information using the YouTube API. It then fetches video IDs from the uploads playlist and collects detailed video metadata and statistics. The collected data is inserted into the MySQL database. If comments are available, the system optionally fetches comments from selected top videos. Finally, a summary is displayed to inform the user about the completed collection process. . This is important because YouTube statistics such as views, likes, comments, subscribers, and thumbnails can change over time. For example, if a user collects data for the same channel again after a few days, the system refreshes the updated view count, like count, comment count, and subscriber count. This keeps the database current while avoiding duplicate records.

For the Dashboard Review page, the sequence starts with channel selection. The user selects a channel from the available stored channels. The system then queries the database and retrieves the required channel and video records. After that, global filters are applied if selected by the user. The filtered data is used for metric calculation. The system calculates KPIs such as average views per video, engagement rate, subscriber-view ratio, and other performance values. Finally, the dashboard interface is rendered with profile details, KPI cards, charts, and video tables. This execution sequence helps the application run in a clear and controlled manner. Each step has a specific role, which makes the system easier to debug, test, and maintain.

4.9 Overall Processing Pipeline

The overall processing pipeline combines all modules of the project into one complete workflow. It connects data collection, storage, filtering, metric calculation, visualization, comparison, and recommendation generation into a single system. The pipeline begins when the user enters YouTube inputs on the Data Collection page.

After collection, the data is cleaned, converted, and stored in the MySQL database. This database acts as the central storage layer of the project. It allows the system to reuse collected data without repeatedly calling the API. When the user opens the dashboard, the system retrieves stored data from MySQL. The user can then apply global filters such as keyword, view range, publish date range, engagement level, and duration group. These filters refine the dataset and make the analysis more targeted.

The filtered data is used to calculate performance metrics. These metrics summarize channel and video performance using values such as average views,

engagement rate, subscriber-view ratio, upload volume, and performance share. The visualization module converts these metrics and records into charts. Charts help users understand trends such as views over time, upload distribution, engagement patterns, top videos, and channel comparison. The comparison module allows multiple channels to be reviewed side by side. It compares subscribers, views, video count, engagement rate, upload volume, and performance share. This helps users understand competitive.

In the future, the report compilation process can be extended by adding export options such as PDF, Word, or Excel reports. A PDF report could be useful for academic submission or formal documentation. A Word report could allow users to edit and customize the content. An Excel report could help users perform further calculations and offline analysis. Such export functionality would allow users to save dashboard . The visualization page shows graphical insights for academic submission, business review, creator planning, client reporting. It would also make the project more professional and practical for real-world use.

Finally, the growth strategy module converts analysis results into useful suggestions. If OpenRouter AI is available, the system generates AI-based recommendations using channel evidence. If AI is unavailable, local rule-based logic provides fallback suggestions. This pipeline makes the application useful for repeated analysis. A channel can be collected, reviewed, compared, filtered, and revisited after the data is refreshed. Since the database updates existing records, changing values such as views, likes, comments, and subscribers can be refreshed without creating duplicate data.

USER INTERFACE & EXPERIENCE

The frontend interface of the YouTube Channel Performance and Engagement Dashboard is built using Streamlit. Streamlit is used because it allows fast development of interactive data applications with simple Python code. The frontend is designed to be simple, clean, and easy to use so that users can understand YouTube analytics without needing technical knowledge.

The interface is organized into multiple sections, and each section performs a specific task. This page-wise structure improves navigation and makes the workflow easy to follow. Users can move from data collection to dashboard review, visualization, comparison, growth strategy, and help section using the sidebar navigation. The layout focuses on simplicity, fast navigation, and clear presentation of analytics. Users interact with the application through different Streamlit components such as sidebar menus, text input fields, buttons, tabs, charts, dataframes, tables, and metric cards. These components make the dashboard interactive and user-friendly.

Each page has a specific purpose. The Data Collection page is used to fetch and store YouTube channel and video information. The Channel Dashboard page presents channel profile details and key performance indicators. The Visualizations page provides charts for views, uploads, and engagement analysis. The Channel Comparison page benchmarks multiple channels side by side. The Growth Strategy page gives AI-powered or rule-based recommendations. The Help and FAQ page explains how to use the application and understand important metrics. This frontend design makes the application suitable for students, creators, analysts, and general users. It reduces manual analysis work and presents complex YouTube data in a simple visual format. [\[4\]](#)

5.1 Data Collection Page

The Data Collection page acts as the starting point of the application. This page is used to collect public YouTube data and store it in the database for further analysis. Since all other pages depend on collected data, this page plays an important role in the overall system. The page provides an input field where users can enter YouTube channel IDs, channel links, handles, usernames, or video links. Users can also enter multiple inputs separated by commas. This makes the page flexible and convenient because users do not need to know only the channel ID.

After entering the input, the user clicks the Fetch and Store Data button. Once the button is pressed, the system starts processing the entered values. It first validates the input and removes empty values. Then it resolves each input into a valid YouTube channel ID. After successful resolution, the system fetches channel information such as channel name, description, thumbnail, subscribers, total views, total videos, and channel creation date. It also fetches video metadata and statistics such as video title, description, publish date, duration, views, likes, comments, and thumbnails.

The collected data is then stored in the MySQL database. If the same channel or video already exists, the system updates the existing record instead of creating duplicates. This keeps the database clean and updated. During data collection, the interface may show progress messages so that users know the system is working. After the process is completed, the page displays a storage summary. This summary helps users understand how many channels and videos were collected or updated. The page also shows recently analyzed channels in a table. This table includes subscribers, total views, total videos, channel creation date, and last analyzed time. It helps users quickly see which channels have already been processed.

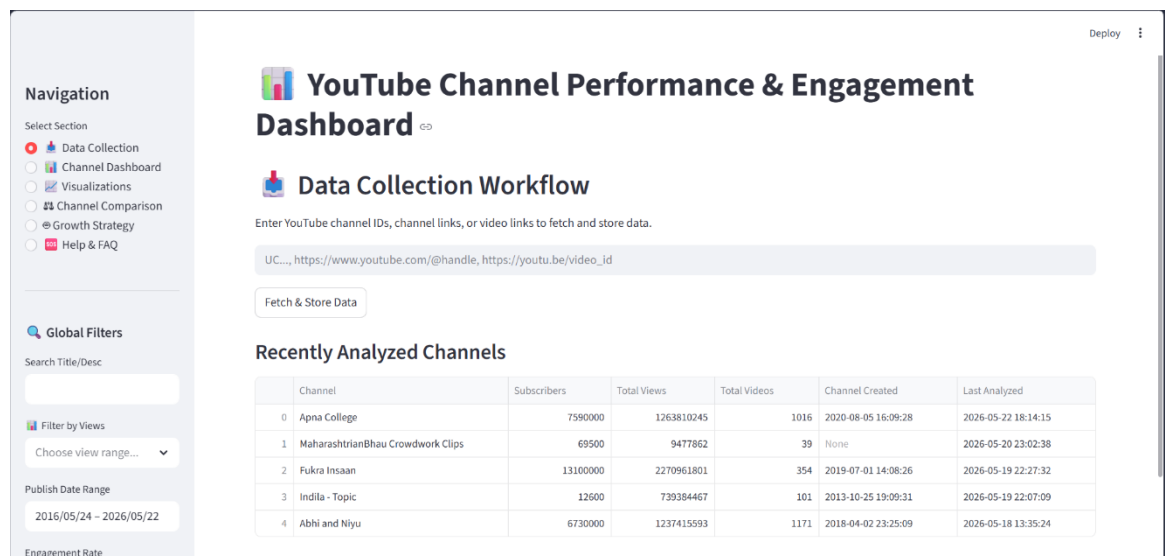


Figure 5.1: Data Collection Page

Overall, the Data Collection page provides a simple and effective way to begin the analysis process. It accepts flexible YouTube inputs, collects useful data, stores it safely, and prepares the information for dashboard, visualization, comparison, and strategy generation pages.

5.2 Channel Dashboard Interface

The Channel Dashboard Interface provides the main performance summary for a selected YouTube channel. After data is collected and stored, users can open this page to review the channel's overall performance in a clear and organized format. This page is useful because it combines channel profile information, calculated metrics, and video-level records in one place. At the top of the page, users can select a channel from the available stored channels. After selecting the channel, the dashboard displays a channel profile card. This card includes the channel thumbnail, channel name, description, subscriber count, total views, total videos, and channel creation date.

The profile card gives users a quick overview of the channel identity and reach. The thumbnail and name help visually identify the channel, while the description explains the channel's content focus. Subscriber count shows the audience size, total views show the overall reach, and total videos show the upload volume. The channel creation date helps users understand how long the channel has been active. Below the profile section, the page displays advanced metric cards. These cards show important calculated values such as average views per video, engagement rate, and subscriber-view ratio. These metrics help users understand performance beyond basic numbers.

Average views per video shows the normal view performance of the channel. Engagement rate shows how actively viewers interact with videos through likes and comments. Subscriber-view ratio shows the relationship between audience size and total reach. These metrics make it easier to compare channels and identify strengths or weaknesses. Below the metric cards, the dashboard includes a paginated video table. This table displays individual video data such as video ID, watch link, thumbnail, title, description, publish date, views, likes, and comments. The watch link allows users to open the video directly on YouTube, while the thumbnail helps users recognize the video visually. [\[6\]](#)

Pagination is used to improve usability when a channel has many videos. Instead of showing all records on one screen, the table is divided into pages. This makes the interface faster, cleaner, and easier to navigate. The profile card gives users a quick overview of the channel identity and reach. The thumbnail and name help visually identify the channel. The watch link allows users to open the video directly on YouTube, while the thumbnail helps users recognize the video visually.

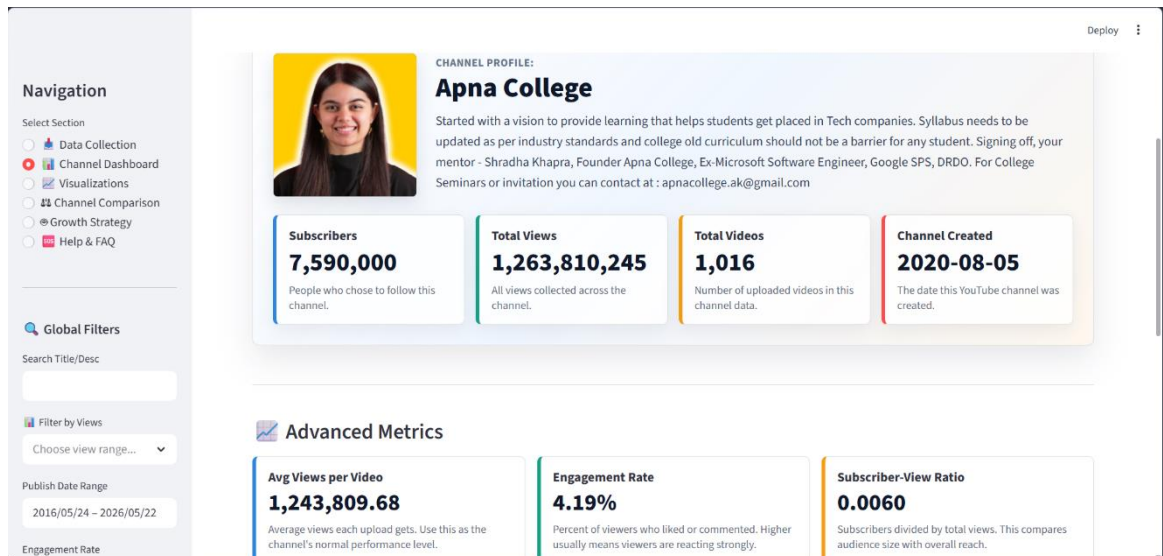


Figure 5.2: Channel Dashboard Page

The Channel Dashboard Interface is important because it provides both summary-level and detailed-level analysis. Users can quickly understand the overall channel performance and also inspect individual video records.

5.3 Visualization Interface

The Visualization Interface allows users to explore YouTube channel data through interactive charts. This page converts stored video statistics into visual insights, making it easier to understand trends, patterns, and performance differences. The page contains tabs for Basic Insights, Advanced Insights, and Custom Metrics. This tab-based structure keeps the interface organized and allows users to focus on one type of analysis at a time.

The Basic Insights tab includes commonly used charts such as views over time, top ten most viewed videos, video distribution by year, and views versus engagement. The views over time chart helps users understand how video performance changes according to publishing dates. The top ten most viewed videos chart highlights the strongest videos of the channel. The video distribution by year chart shows upload activity across different years. The views versus engagement chart helps compare reach and audience interaction.

The Advanced Insights tab provides deeper analysis. It includes an upload heatmap, view distribution histogram, engagement multi-line chart, and engagement funnel. The upload heatmap shows upload activity by day and time, helping users understand posting patterns. The view distribution histogram shows how views are

spread across videos. The engagement multi-line chart compares likes, comments, and engagement trends. The engagement funnel shows the flow from views to likes to comments, helping users understand audience response.

The Custom Metrics tab allows users to select specific metrics such as views, likes, or comments and analyze them across daily, monthly, or yearly periods. This gives users more control over the analysis. For example, a user can check monthly views, yearly likes, or daily comment trends depending on their requirement. The Visualization Interface uses interactive charts, which allow users to explore values through hover details and dynamic rendering. This makes the charts more useful than static images because users can inspect exact values and understand data points clearly.

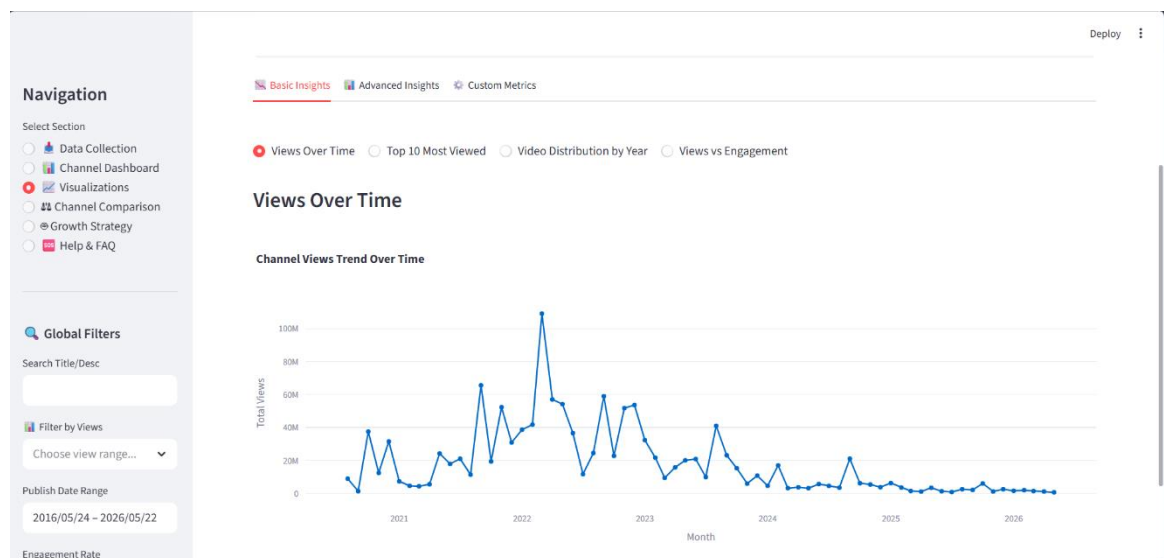


Figure 5.3: Visualization Page

Overall, the Visualization Interface helps users understand YouTube performance visually. It supports trend analysis, engagement analysis, upload pattern analysis, and custom metric exploration. This makes the charts more useful than static images because users can inspect exact values and understand data points clearly.

5.4 Channel Comparison Interface

The Channel Comparison Interface helps users compare multiple YouTube channels side by side. This page is useful when users want to understand how one channel performs compared to other channels in the same category or niche. Instead of analyzing only one channel, users can select multiple channels and view their performance together. The interface supports smart search by channel name, direct channel link, handle, video link, or channel ID. This makes channel selection easier

because users do not need to remember only the channel ID. They can search using the format that is most convenient for them.

The comparison list is limited to five channels. This limit is useful because comparing too many channels at once can make charts crowded and difficult to read. By limiting the selection, the interface keeps the visual analysis clean, readable, and meaningful. Once at least two channels are selected, the page displays selected channel cards. These cards show basic details of the selected channels and help users confirm which channels are being compared. After that, the system generates a leaderboard ranking based on performance values.

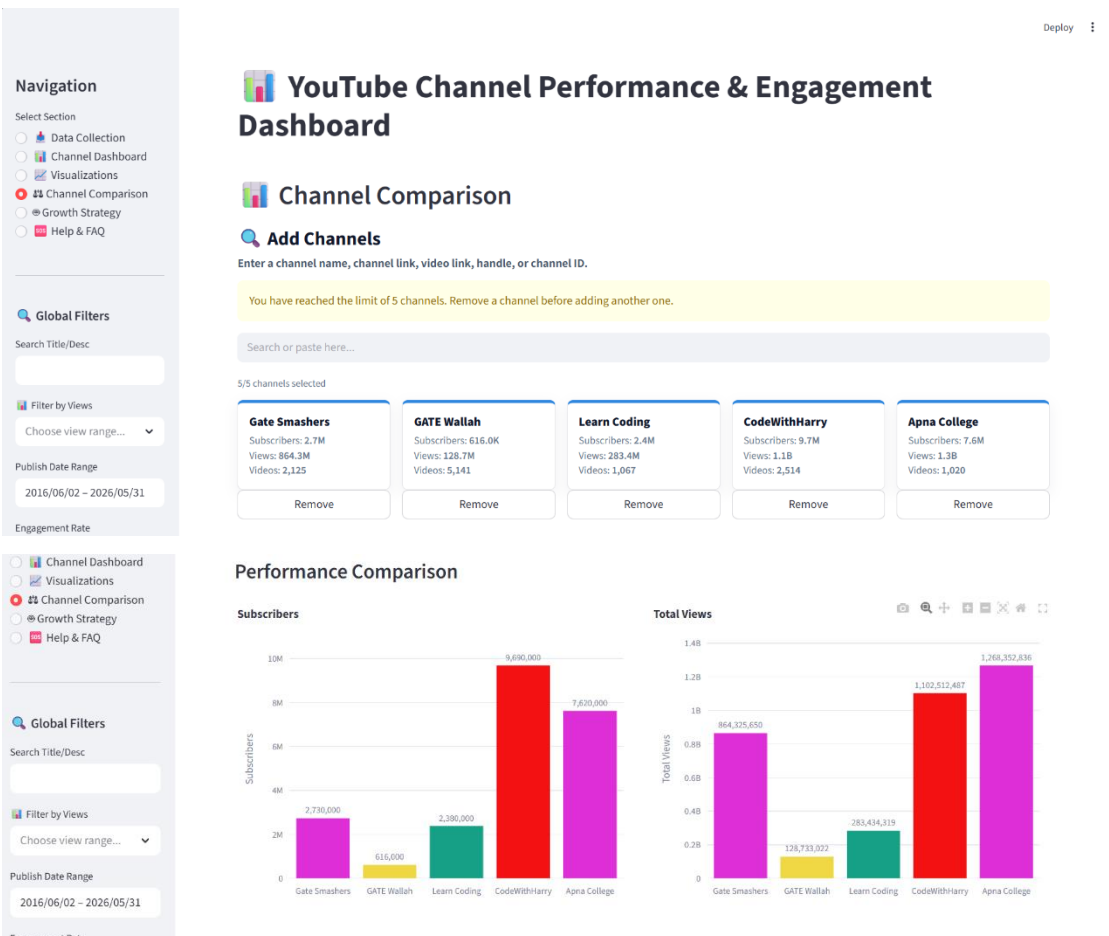


Figure 5.4: Channel Comparison Page

The benchmark analysis table compares important metrics such as subscribers, total views, total videos, average views, engagement rate, and upload volume. This helps users identify which channel has better audience size, stronger reach, higher activity, or better engagement. The interface also includes multiple charts such as subscriber chart, total views chart, upload volume chart, and performance share pie

chart. The subscriber chart compares audience size, while the total views chart compares overall reach. The upload volume chart shows how active each channel is in uploading videos. The performance share pie chart shows each channel's contribution in the selected comparison group.

This limit is useful because comparing too many channels at once can make charts crowded and difficult to read. By limiting the selection, the interface keeps the visual analysis clean, readable, and meaningful. Once at least two channels are selected, the page displays selected channel cards. These cards show basic details of the selected channels and help users confirm which channels are being compared. After that, the system generates a leaderboard ranking based on performance values.

Overall, the Channel Comparison Interface helps users perform competitive analysis in a simple and visual way. It allows creators, students, and analysts to identify strengths, weaknesses, and performance gaps between channels.

5.5 Global Filter Interface

The Global Filter Interface is placed in the sidebar and affects the dashboard and visualization views. This design makes the filters easily accessible from different analysis pages. Users can apply filters once and immediately see the updated results in charts, tables, and metrics. The filter interface allows users to search by video title or description. This is useful when users want to analyze videos related to a specific topic or keyword. For example, if a channel uploads different types of content, the user can search for one topic and analyze only those videos. Users can also select view ranges to classify videos based on performance. View categories may include Low, Medium, High, and Viral. This helps users focus on specific performance groups, such as only viral videos or low-performing videos.

The publish date range filter allows users to analyze videos uploaded during a selected time period. This is useful for monthly, yearly, or campaign-based analysis. Users can compare performance before and after a specific date range. The engagement level filter allows users to view videos with Low, Medium, or High engagement. Engagement is calculated using likes and comments relative to views. This helps users identify videos where viewers interacted more actively.

The duration filter classifies videos based on length, such as Short, Medium, or Long. This helps users understand how video duration affects views and engagement. A Clear All Filters button is provided to reset the filter state. This improves usability

because users can quickly return from a narrow analysis to the complete dataset without manually changing each filter.

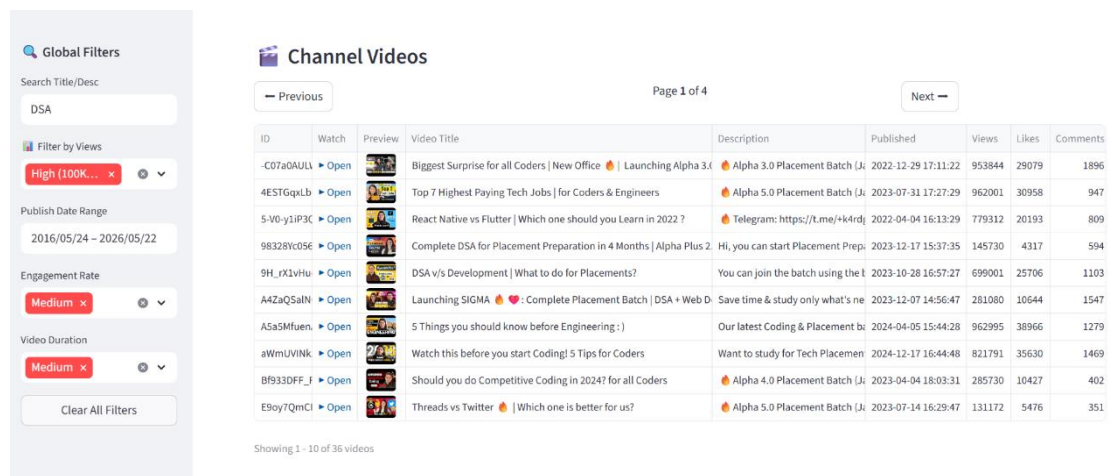


Figure 5.5: Global Filter Sidebar

Overall, the Global Filter Interface makes the dashboard more flexible and interactive. It helps users perform focused analysis and quickly switch between filtered and complete views.

5.6 Growth Strategy Interface

The Growth Strategy Interface presents channel-level performance analysis and recommendation-based guidance for improving YouTube channel growth. This page is designed to help users understand not only what has happened in the channel's past performance, but also what actions can be taken for future improvement. At the top of the page, the interface displays important summary values such as total videos, average views, and total comments. These values give users a quick idea of the channel's content volume, normal view performance, and audience response level.

The page also displays charts for high-performing and low-performing videos. These videos are identified using view-count quantiles. Videos above the higher quantile are treated as strong-performing videos, while videos below the lower quantile are treated as weak-performing videos.

The strategy section displays content insights such as what works and what needs improvement. For example, the system may identify that certain topics, title patterns, or video types are performing better than others.

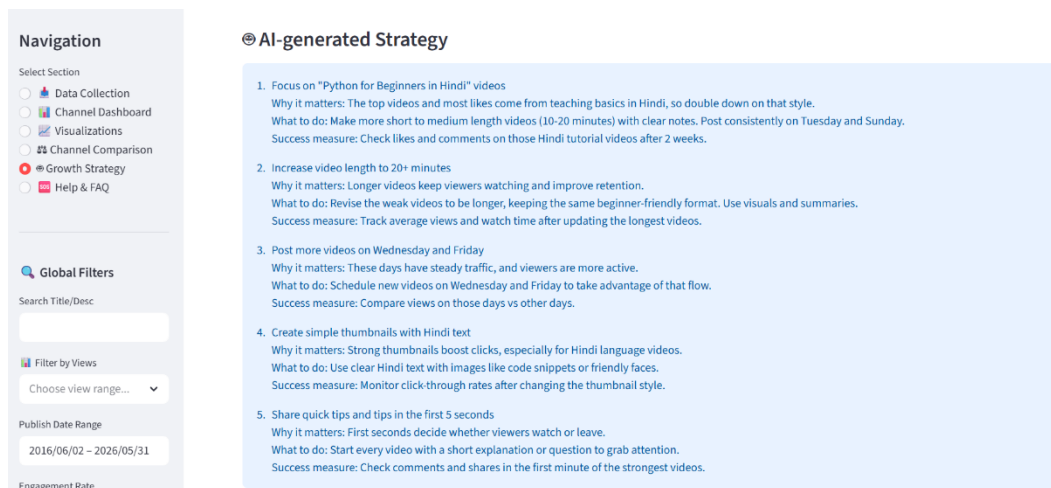


Figure 5.6: Growth Strategy Page

If OpenRouter is configured, the system displays AI-generated strategy text. The AI uses channel evidence such as top videos, weak videos, engagement values, upload patterns, and extracted topics to generate suggestions. These suggestions may include content ideas, title improvement tips, upload frequency advice, engagement improvement methods, and future topic recommendations. If OpenRouter is not configured or the API fails, the system shows local rule-based recommendations. This ensures that users still receive useful guidance even when AI access is unavailable. The local recommendations are generated from stored data and simple performance rules. Overall, the Growth Strategy Interface makes the dashboard more action-oriented. It helps users move from analysis to decision-making by providing practical suggestions for channel improvement.

5.7 Help and FAQ Interface

The Help and FAQ Interface explains how to use every page in the application. This page is useful for new users because it provides guidance about the dashboard workflow, features, and metrics. The Help section describes the purpose of major pages such as Data Collection, Channel Dashboard, Visualizations, Channel Comparison, and Growth Strategy. It explains what each page does and how users can interact with it. The page also explains common YouTube analytics metrics such as views, likes, comments, engagement rate, average views, upload volume, subscriber-view ratio, and performance share. This helps users understand the meaning of the numbers shown in the dashboard. [\[7\]](#)

The FAQ section answers typical user questions. It explains why a channel may not be found, why comments or likes may be missing, how to refresh data, why charts

may appear empty, and why channel comparison is limited to five channels. For example, a channel may not be found if the input link is invalid, the handle is incorrect, or the channel is unavailable. Comments may be missing if comments are disabled on a video. Likes may be unavailable if YouTube does not return that value. Charts may appear empty if there is no collected data or if filters remove all records.

The Help and FAQ page improves user understanding and reduces confusion. It allows users to solve common issues without needing external support. This makes the application more user-friendly and complete.

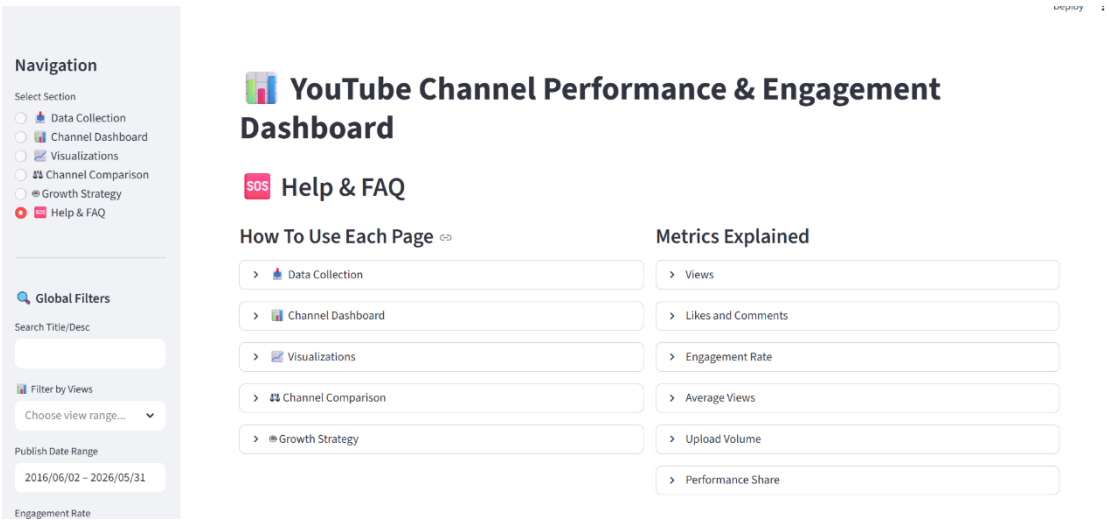


Figure 5.7: Help and FAQ Page

5.8 Dashboard Interface

The Dashboard Interface combines all pages of the project into a single Streamlit application. It acts as the main workspace where users can collect, analyze, compare, visualize, and review YouTube channel performance data. The interface is designed in a modular way so that each page performs a specific function while still remaining connected to the overall system.

The user can navigate between different modules through the sidebar navigation menu. This sidebar allows quick movement between Data Collection, Channel Dashboard, Visualization, Channel Comparison, Growth Strategy, and Help and FAQ pages. This makes the application easy to use because users do not need to open separate tools for different tasks. The title of the application clearly identifies it as a YouTube Channel Performance and Engagement Dashboard. This helps users immediately understand the purpose of the system. The layout is simple and organized, which makes

it suitable for academic demonstration as well as practical channel review. The interface uses different visual and interactive elements such as metric cards, data tables, image thumbnails, charts, tabs, progress indicators, warning messages, success messages, and expandable help sections. Metric cards are used to show important values such as subscribers, total views, average views, and engagement rate.

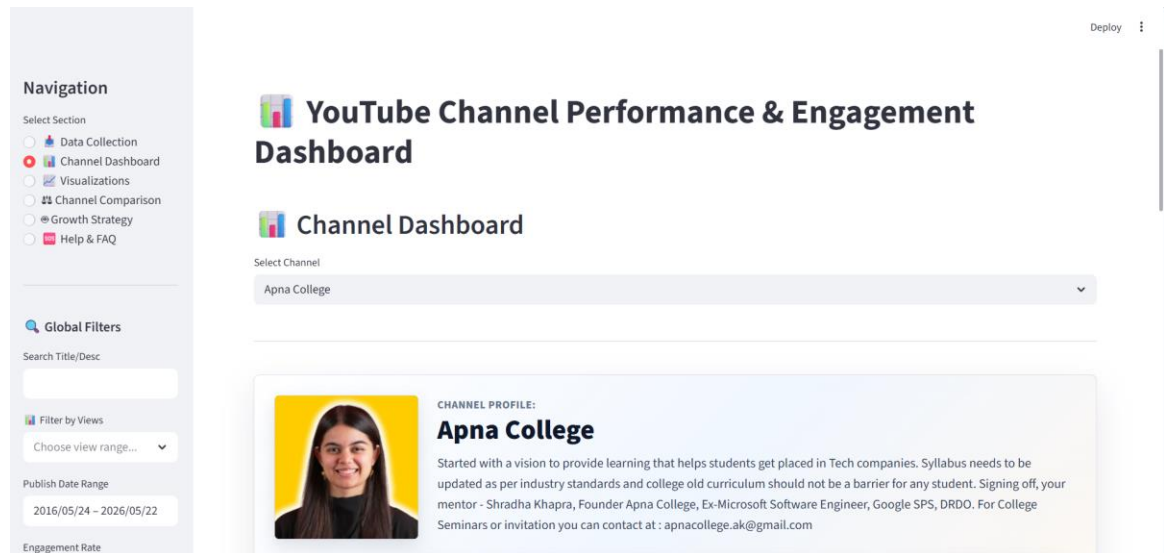


Figure 5.8: Complete Dashboard Interface

Charts are used to present trends and comparisons in a more understandable way. Tabs divide large sections into smaller parts, making the interface cleaner. Progress indicators show users that data collection is running. Warning and success messages provide feedback about errors, missing data, or completed operations. Expandable help sections explain features without overcrowding the screen. The dashboard also keeps filter state available across relevant pages. This means users can apply filters such as keyword search, view range, date range, engagement level, or duration group and continue analyzing the refined data in dashboard and visualization sections. This improves consistency and makes the analysis more focused.

CONCLUSION

The YouTube Channel Performance and Engagement Dashboard provides a practical and effective solution for collecting, storing, analyzing, and visualizing YouTube channel data. The project combines Python programming, YouTube API integration, MySQL database storage, Streamlit-based interface design, Plotly visualizations, and AI-assisted strategy generation into one complete working system.

The dashboard helps users understand important channel metrics such as subscribers, total views, total videos, average views per video, engagement rate, subscriber-view ratio, upload frequency, and performance share. It also supports detailed video-level analysis through interactive tables, charts, filters, and comparison tools. This project demonstrates how public YouTube data can be converted into meaningful insights through structured data processing. Features such as multiple input support, global filters, channel comparison, custom metric trends, and growth recommendations make the application more useful than a simple statistics viewer. The system not only displays data but also helps users interpret performance and identify areas for improvement.

Overall, the project strengthened practical knowledge of Python development, API-based data collection, relational database operations, dashboard design, data visualization, and AI-supported analysis. It also improved understanding of how real-world analytics systems collect, process, store, and present data to users.

In the future, the system can be enhanced by adding comment sentiment analysis, automated PDF/Excel report export, scheduled data refresh, predictive performance modeling, and richer creator-focused recommendations. These improvements would make the dashboard more powerful, professional, and useful for content creators, analysts, and academic users.

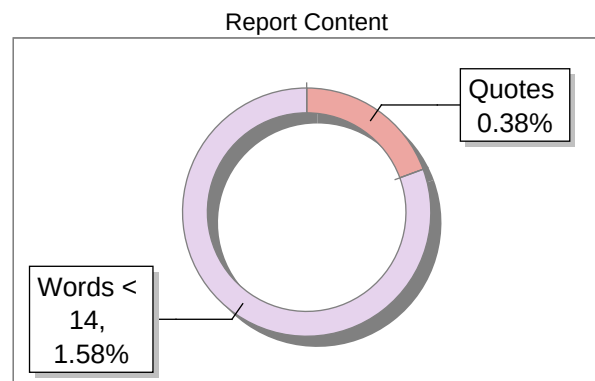
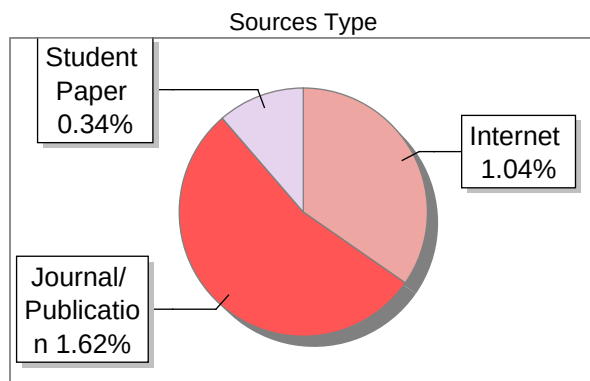
REFERENCES

- [1] W. McKinney, Python for Data Analysis, 3rd ed., Sebastopol, CA, USA: O'Reilly Media, 2022.
- [2] J. VanderPlas, Python Data Science Handbook, 2nd ed., Sebastopol, CA, USA: O'Reilly Media, 2022.
- [3] A. Downey, Think Python: How to Think Like a Computer Scientist, 2nd ed., Sebastopol, CA, USA: O'Reilly Media, 2015.
- [4] L. Khan and A. Malik, “Researching YouTube: Methods, Tools, Analytics,” in The SAGE Handbook of Social Media Research Methods, 2nd ed., London, U.K.: SAGE Publications, 2022, pp. 651–663.
- [5] S. Stieglitz, M. Mirbabaie, B. Ross, and C. Neuberger, “Social media analytics—Challenges in topic discovery, data collection, and data preparation,” International Journal of Information Management, vol. 39, pp. 156–168, 2018.
- [6] B. Batrinca and P. C. Treleaven, “Social media analytics: A survey of techniques, tools and platforms,” AI & Society, vol. 30, no. 1, pp. 89–116, 2015.
- [7] T. Le, B. H. T. Nguyen, M. V. Nguyen-Thi, and T. Le, “EnTube: A dataset for YouTube video engagement analytics,” 2022.
- [8] W. Hoiles, A. Aprem, and V. Krishnamurthy, “Engagement and popularity dynamics of YouTube videos and sensitivity to meta-data,” IEEE Transactions on Knowledge and Data Engineering, vol. 29, no. 7, pp. 1426–1437, 2017.
- [9] M. Park, M. Naaman, and J. Berger, “A data-driven study of view duration on YouTube,” 2016.
- [10] A. Deubel, J. Breuer, J. Kohne, and M. R. Mohseni, “Overview of working with data from YouTube,” GESIS Guides to Digital Behavioral Data, 2024.

Submission Information

Author Name	RAJLAXMI KALASKAR
Title	final_report
Paper/Submission ID	5761833
Submitted by	rajurkar_am@mgmcen.ac.in
Submission Date	2026-05-30
Total Pages, Total Words	82, 23122
Document type	Project Report

Result Information

Similarity **3 %**

Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

